IMAGE ENCRYPTION AND DECRYPTION IN FPGA USING CHAOTIC ALGORITHM AND SYNTHESIS IN CADENCE

A MINI PROJECT REPORT

Submitted by

ADITHYAN V (TCR23EC003)

ASWIN (TCR23EC013)

HRIDHYA M (TCR23EC031)

SREELAKSHMI K (TCR23EC062)

to

the APJ Abdul Kalam Technological University

in partial fulfillment of the requirements for the award of the Degree

of

Bachelor of Technology

in

Electronics and Communication Engineering

DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING

GOVERNMENT ENGINEERING COLLEGE THRISSUR

KERALA

31 MARCH 2026

DECLARATION

We,the undersigned, hereby declare that the mini project report (Image Encryption and Decryption in FPGA using Chaotic Algorithm and Synthesis in Cadence), submitted for partial fulfillment of the requirements for the award of degree of Bachelor of Technology of the APJ Abdul Kalam Technological University, Kerala is a bonafide work done by us under supervision of Riyas K S, Assistant Professor, Government Engineering College, Thrissur. This submission represents our ideas in our own words and where ideas or words of others have been included; we have adequately and accurately cited and referenced the original sources. We also declare that we have adhered to ethics of academic honesty and integrity and have not misrepresented or fabricated any data or idea or fact or source in our submission. We understand that any violation of the above will be a cause for disciplinary action by the institute and/or the University and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been obtained. This report has not previously formed the basis for the award of any degree, diploma or similar title of any other University.

GEC THRISSUR
Date : 31-03-2026

Adithyan V
Aswin
Hridhya M
Sreelakshmi K

ELECTRONICS AND COMMUNICATION ENGINEERING

GOVERNMENT ENGINEERING COLLEGE THRISSUR
THRISSUR-680 009

CERTIFICATE

This is to certify that this is a bonafide record of the Project Work titled "Image Encryption and Decryption in FPGA using Chaotic Algorithm and Synthesis in Cadence" done by

Adithyan V	**Reg No. TCR23EC003**
Aswin	**Reg No. TCR23EC013**
Hridhya M	**Reg No. TCR23EC031**
Sreelakshmi K	**Reg No. TCR23EC062**

of third year B. Tech. Electronics and Communication Engineering in partial fulfilment of the requirement for the award of ***Bachelor of Technology Degree in Electronics and Communication Engineering*** *under the* ***A. P. J. Abdul Kalam Technological University*** *for the year 2026.*

Guide:

Prof. Riyas K S
Assistant Professor
Dept. of Electronics Engg.
Government Engineering College
Thrissur

Project Coordinator:

Prof. Sunny T D
Assistant Professor
Dept. of Electronics Engg.
Government Engineering College
Thrissur

Dr. Asha J
Assistant Professor
Dept. of Electronics Engg.
Government Engineering College
Thrissur

ACKNOWLEDGMENT

We wish to record our indebtedness and thankfulness to all those who helped us prepare this Mini project titled Image Encryption and Decryption in FPGA using Chaotic Algorithm and Synthesis in Cadence and present it in a satisfactory way. First and foremost we thank Dr.Roy , Head of the Department, Dept of Electronics and Communication Engineering for giving us the opportunity to present this project. We extend our gratitude to Prof. Riyas K S, Assistant Professor, Dept of Electronics and Communication Engineering for his valuable guidance, encouragement and co-operation during the course of this project and its presentation.His support played a key role in the successful completion of this project. Finally, we would like to extend our sincere gratitude towards teachers, friends of Dept of Electronics and Communication Engineering and our family members who have always been helpful.

GEC THRISSUR
Date : 31-03-2026

Adithyan V
Aswin
Hridhya M
Sreelakshmi K

ABSTRACT

With the rapid growth of digital communication and multimedia applications, secure transmission and storage of image data have become an important challenge. Conventional software-based encryption methods are often vulnerable to cyber-attacks and may not provide sufficient real-time performance for high-speed applications. To address these limitations, this project focuses on the design and implementation of a hardware-based image encryption and decryption system using a Field Programmable Gate Array (FPGA).

The proposed system utilizes a chaotic pseudo-random number generator based on the logistic map to generate highly unpredictable key sequences. These chaotic sequences are combined with grayscale image pixel data using XOR operations to achieve image encryption. The same chaotic key is reused during decryption to accurately recover the original image, exploiting the reversible property of the XOR operation. The complete system is implemented using Verilog HDL on a Nexys FPGA board, employing fixed-point arithmetic to efficiently realize the chaotic map in hardware.

The encrypted and decrypted image data are verified through simulation and hardware testing, ensuring correct functionality and synchronization between pixel data and chaotic keys.This project results in a secure, high-speed, and resource-efficient image encryption system and highlights the effectiveness of chaos-based encryption techniques for real-time image security applications.

Contents

CHAPTER 1: INTRODUCTION **21**

CHAPTER 2: LITERATURE SURVEY AND THEORY **23**

2.1 OVERVIEW OF IMAGE SECURITY 23
2.2 CHAOTIC SYSTEMS IN CRYPTOGRAPHY 23
2.3 THE LOGISTIC MAP AND 2D SYSTEMS 23
2.4 Bifurcation Diagram of Logistic Map 24
2.5 FIXED-POINT ARITHMETIC IN FPGA 24
2.6 FIELD PROGRAMMABLE GATE ARRAYS (FPGA) 25

CHAPTER 3:PROPOSED SYSTEM **27**

3.1 SYSTEM ARCHITECTURE 27
3.2 ENCRYPTION (CORE) SUBSYSTEM 29
3.2.1 Chaotic Map Implementation (Logistic Core) 30
3.2.2 Nonlinear Coupling Logic 30
3.2.3 Keystream Extraction Logic (XOR-based Diffusion) . . 31
3.3 MEMORY MANAGEMENT 31
3.3.1 Dual-Port RAM Usage 31
3.3.2 Handling 2500-Pixel Data Arrays 32
3.4 FIFO BASED BUFFERING 32
3.5 FINITE STATE MACHINE DESIGN 33
3.5.1 Phase 1: Encryption State 33
3.5.2 Phase 2: Decryption State 33
3.6 COMMUNICATION INTERFACE 34
3.6.1 UART Transmitter Design 34
3.7 RTL SCHEMATIC AND HARDWARE INTEGRATION . . . 34
3.7.1 Input and Control Signal Routing 35

3.7.2 Memory Interface and Address Generation 35
3.7.3 Chaotic Sequence Generator 35
3.7.4 Data Conversion and Scaling 36
3.7.5 XOR-Based Encryption and Decryption 36
3.7.6 FIFO Buffer and Data Flow Control 36
3.7.7 UART Transmission Interface 37
3.7.8 Overall Data Flow 37
3.7.9 Conclusion 37
3.8 RESULTS 38
3.9 CONCLUSION OF PROPOSED SYSTEM 39

CHAPTER 4: HARDWARE IMPLEMENTATION 41
4.1 INTRODUCTION 41
4.2 TARGET HARDWARE PLATFORM 41
4.3 DESIGN FLOW 41
4.4 HARDWARE ARCHITECTURE 42
4.4.1 Chaotic Sequence Generator 42
4.4.2 Encryption Module 42
4.4.3 Memory Implementation 42
4.4.4 FIFO Buffer 42
4.4.5 Control Unit(FSM) 43
4.4.6 UART Communication Module 43
4.5 TOP LEVEL INTEGRATION 43
4.6 RTL DESIGN AND RESOURCE UTILIZATION 44
4.7 HARDWARE TESTING AND VALIDATION 44
4.8 ENTROPY ANALYSIS 44

CHAPTER 5: HDL IMPLEMENTATION 47
5.1 Subsystem.v 47
5.2 `HDL_FIFO` 50
5.3 `SimpleDualPortRAM_generic`.v 56
5.4 `veri_ency`.v 58
5.5 `S_R_Flip_Flop`.v 59
5.6 `uart_tx`.v 62
5.7 `top`.v 65

5.8 Subsystem_tb.v 68
5.9 nexys.xdc 72
5.10 check_data_and_save.py 73

CHAPTER 6: CADENCE GENUS SYNTHESIS 77
6.1 CADENCE GENUS 77
6.2 GENUS SYNTHESIS PROCESS 77
6.3 GENUS_RUN.TCL SCRIPTING 78
6.4 ANALYSIS REPORTS 82
6.4.1 Timing Report 82
6.4.2 Power Analysis 83
6.4.3 Area Report 84
6.4.4 Gate Count Report 86
6.4.5 Netlist Report 86

CHAPTER 7: CADENCE INNOVUS IMPLEMENTATION 89
7.1 CADENCE INNOVUS 89
7.2 CADENCE IMPLEMENTATION PROCESS 89
7.3 INNOVUS_RUN.TCL 90

CHAPTER 8: CONCLUSION 95

List of Figures

2.1 Bifurcation Diagram of Logistic Map for 1000 iterations and x=0.5 24
2.2 Nexys 4DDR Artix 7 By Analog Devices 25
3.1 Flowchart of Encryption Process 28
3.2 Flowchart of Decryption Process 29
3.3 Block Diagram of Top Module 29
3.4 Block Diagram of Encryption Subsystem 30
3.5 RTL Schematic of Entire System Generated in Vivado 34
3.6 RTL Schematic of Top Module Generated in Vivado 38
3.7 Original, Encrypted and Decrypted Image 39
4.1 Entropy Comparison of Original, Encrypted, and Decrypted Images 45
6.1 Cadence-Genus Schematic of Image Encryption using Chaotic Algorithm 78
6.2 Timing Analysis of Critical Paths from Cadence Genus . . . 83
6.3 Power Details report 84
6.4 Power Breakdown of Top Module from Cadence Genus . . . 84
6.5 Data Path Area Report 85
6.6 Area Distribution of Top Module and UART Submodule from Cadence Genus 85
6.7 Gate Count Report 86
6.8 Netlist Report 87
7.1 Cadence-Innovus Implemented Design of Image Encryption using Chaotic Algorithm 90

List of Tables

6.1 Timing Report Summary 83
6.2 Power Consumption Summary 84
6.3 Area Report 85
6.4 Design Summary from Cadence Analysis 87

ABBREVIATIONS

- **RTL** – Register Transfer Level
- **PPA** – Power, Performance, Area
- **TCL** – Tool Command Language
- **SDC** – Synopsys Design Constraints
- **LEF** – Library Exchange Format
- **LIB** – Liberty Library File
- **SDF** – Standard Delay Format
- **SPEF** – Standard Parasitic Exchange Format
- **DEF** – Design Exchange Format
- **FPGA** – Field Programmable Gate Array
- **ASIC** – Application-Specific Integrated Circuit
- **UART** – Universal Asynchronous Receiver Transmitter
- **PRNG** – Pseudo Random Number Generator
- **MHz** – Megahertz
- **ns** – Nanoseconds
- **ps** – Picoseconds
- μm^2 – Square Micrometers
- **GPDK** – Generic Process Design Kit
- **UMC** – United Microelectronics Corporation (foundry reference)

CHAPTER 1
INTRODUCTION

With the rapid advancement of digital communication systems and the increasing reliance on multimedia data, ensuring the security of digital images has become a critical challenge. Images are widely used in applications such as medical diagnostics, surveillance systems, military communication, and remote sensing, where confidentiality and integrity are of utmost importance. However, unlike textual data, digital images exhibit unique characteristics such as high redundancy, large data size, and strong correlation among adjacent pixels, which make traditional encryption algorithms less efficient and computationally expensive for real-time applications.

Conventional encryption standards such as Data Encryption Standard (DES) and Advanced Encryption Standard (AES), although highly secure for textual data, are not optimized for image processing due to their sequential nature and high computational overhead. These limitations have led to the exploration of alternative techniques that can provide both high security and computational efficiency, especially for hardware-based implementations.

In this context, chaos-based cryptography has emerged as a promising solution for image encryption. Chaotic systems are nonlinear dynamical systems that exhibit properties such as sensitivity to initial conditions, non-periodicity, ergodicity, and pseudo-random behavior. These characteristics closely align with the requirements of cryptographic systems, where unpredictability and diffusion are essential for ensuring data security.

One of the most widely used chaotic models in cryptographic applications is the logistic map, defined as:

$$x_{n+1} = r \cdot x_n(1 - x_n) \tag{1}$$

The logistic map is favored due to its simplicity and its ability to generate complex, random-like sequences using basic arithmetic operations. However, when implemented in digital systems with finite precision, chaotic maps may suffer from short periodic cycles and reduced randomness.

To address these limitations, enhanced chaos-based approaches have been proposed, including the use of higher-dimensional chaotic systems and dynamic parameter variations.

In this project, a chaos-based image encryption and decryption system is implemented on a Field Programmable Gate Array (FPGA), leveraging the advantages of hardware-level parallelism and high-speed processing.

A key design consideration in implementing chaotic systems on FPGA is the use of fixed-point arithmetic. In this project, a Q2.30 format is employed to achieve high precision while minimizing hardware complexity.

The core of the system is a high-precision two-dimensional chaotic generator that produces a pseudo-random keystream used for encryption. The keystream is combined with input image pixels using a bitwise XOR operation.

To support efficient data flow, the system incorporates memory modules, FIFO buffers, and UART communication. A modular HDL-based architecture ensures scalability and efficient implementation.

In summary, this project presents an efficient FPGA-based image encryption system that combines chaos theory with hardware acceleration to achieve high performance, strong security, and real-time capability.

CHAPTER 2
LITERATURE SURVEY AND THEORY

2.1 OVERVIEW OF IMAGE SECURITY

With the rapid advancement of network technology, the security of digital images has become paramount. Unlike textual data, digital images possess intrinsic characteristics such as high redundancy, large data volumes, and strong correlation between adjacent pixels. Standard encryption algorithms like Data Encryption Standard (DES) and Advanced Encryption Standard (AES) are often computationally expensive when applied to high-resolution images. This has led to the exploration of chaos-based cryptography as a more efficient alternative for hardware implementation.

2.2 CHAOTIC SYSTEMS IN CRYPTOGRAPHY

Chaotic systems are nonlinear dynamical systems that are highly sensitive to initial conditions—a property often termed as the ”Butterfly Effect.” In cryptography, these systems are utilized as Pseudo-Random Number Generators (PRNGs). The core advantages include:

- **Non-Periodicity:** Chaotic orbits do not repeat, providing a high degree of randomness.
- **Ergodicity:** The system variables cover the entire phase space over time, ensuring diffusion.
- **Key Sensitivity:** Even a 10^{-15} change in the initial seed results in a completely different encrypted output.

2.3 THE LOGISTIC MAP AND 2D SYSTEMS

The Logistic Map is a 1D chaotic system defined by the equation $x_{n+1} = r \cdot x_n(1 - x_n)$. While simple, 1D maps are vulnerable to phase space reconstruction

attacks. To enhance security, this project utilizes a 2D Chaotic System involving nonlinear coupling. This increases the complexity of the keystream and expands the key space significantly, making brute-force attacks computationally infeasible.

2.4 Bifurcation Diagram of Logistic Map

The Logistic Map is a one-dimensional chaotic system defined by the equation $x_{n+1} = r \cdot x_n(1 - x_n)$, which generates sequences exhibiting random-like behavior for certain values of the control parameter r. Due to its simplicity and ability to produce chaotic outputs using basic arithmetic operations, it is widely used in image encryption applications. The generated chaotic sequence is typically normalized and used to create a noise image, which is then combined with the original image using logical operations such as XOR to achieve encryption.

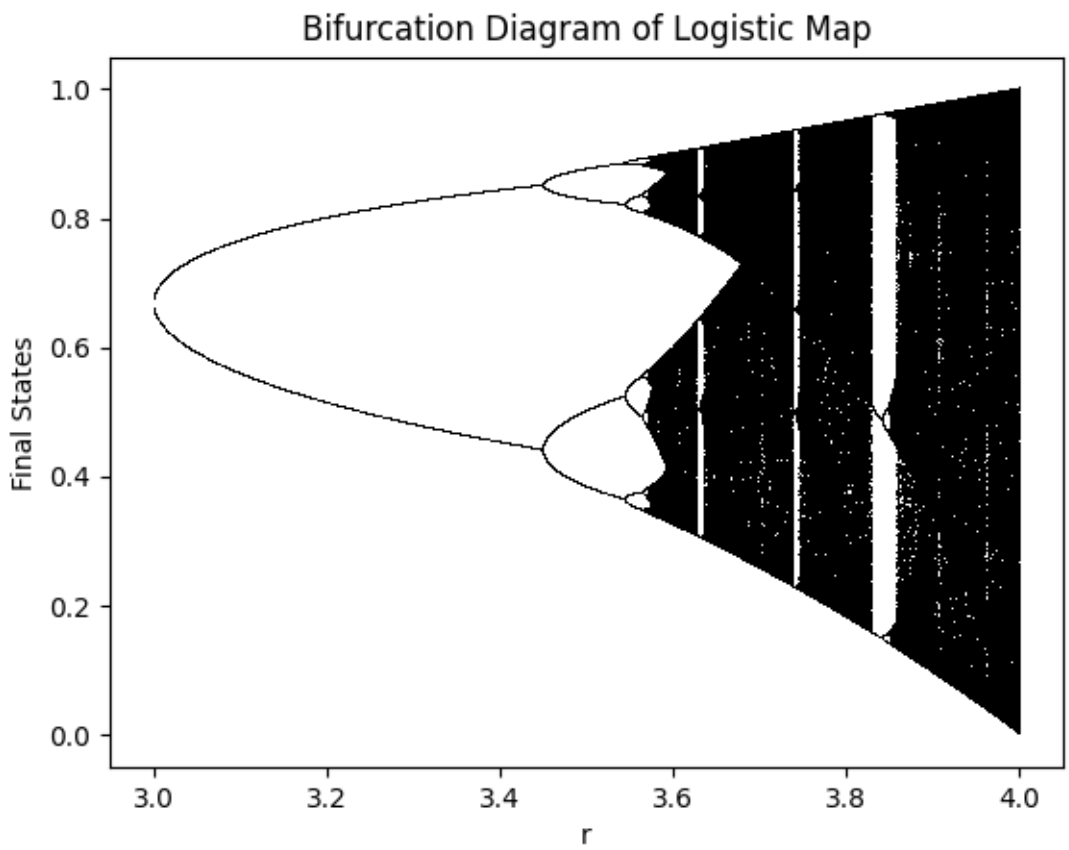


Figure 2.1: Bifurcation Diagram of Logistic Map for 1000 iterations and x=0.5

2.5 FIXED-POINT ARITHMETIC IN FPGA

Chaotic equations involve heavy floating-point multiplications. However, FPGAs are

most efficient at performing integer or fixed-point arithmetic. In this project, the **Q2.30 format** is employed. This represents numbers using 32 bits, where 2 bits are reserved for the integer part and 30 bits for the fractional part. This provides the high precision necessary to maintain chaotic properties without the resource overhead of a Floating Point Unit (FPU).

2.6 FIELD PROGRAMMABLE GATE ARRAYS (FPGA)

FPGAs provide the ideal platform for image encryption due to their parallel processing capabilities. While a CPU processes pixels sequentially, an FPGA can be architected to perform chaotic iterations and pixel XORing in a pipelined fashion, drastically reducing latency and enabling real-time encryption of video streams or high-resolution imagery.//

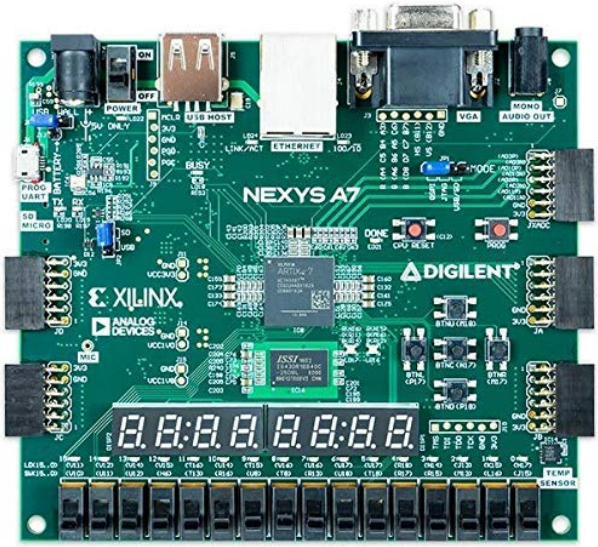


Figure 2.2: Nexys 4DDR Artix 7 By Analog Devices

CHAPTER 3
PROPOSED SYSTEM

3.1 SYSTEM ARCHITECTURE

The proposed system implements a complete FPGA-based image encryption and decryption architecture using a chaotic pseudo-random number generator (PRNG). The design is targeted for the Artix-7 Nexys DDR FPGA board and supports end-to-end processing including encryption, transmission, and decryption.

The system was initially modeled and verified in MATLAB Simulink using HDL-compatible blocks to ensure hardware feasibility. The verified design was then converted into synthesizable Verilog code and implemented using Xilinx Vivado. The encrypted pixel stream generated by the FPGA is transmitted through a UART interface and received on a host system using Python. The same encrypted data is also looped back into the FPGA for decryption using identical keystream generation, thereby reconstructing the original image. The top-level architecture consists of the following major components:

- Pixel input interface (memory-based input)
- Encryption core (chaotic PRNG + XOR diffusion)
- Memory modules (RAM and buffers)
- UART communication interface
- Decryption module (XOR with synchronized keystream)
- Control logic and clock management

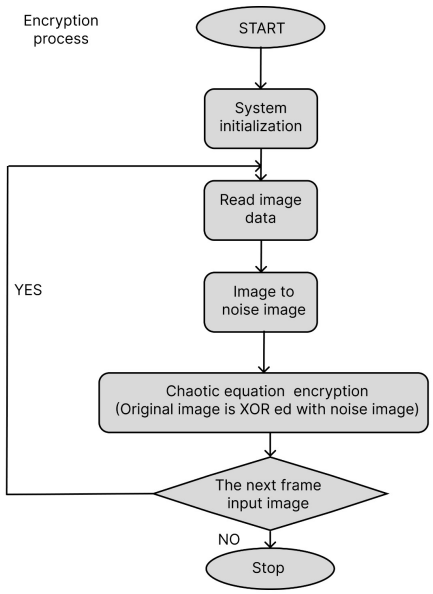


Figure 3.1: Flowchart of Encryption Process

The overall operational flow of the proposed encryption system is illustrated in Fig.3.1. The process begins with system initialization, where all internal registers, memory units, and control signals are configured to their initial states. Once initialized, the system proceeds to read the image data, which is stored in memory as pixel values

The input image is then transformed into a noise-like representation using a chaotic pseudo-random number generator. This stage introduces randomness into the system and generates a keystream that is highly sensitive to initial conditions. The original image pixels are subsequently combined with the generated noise sequence using a bitwise XOR operation. This operation produces the encrypted image, ensuring both confusion and diffusion properties required for secure encryption.

The decryption process of the proposed system is illustrated in Fig.3.2 The process begins with system initialization, after which the encrypted image is read from memory or input source. This encrypted data corresponds to the output generated during the encryption stage and is assumed to be available in stored form rather than as a continuous real-time stream.

The core of the decryption process involves the application of the same chaotic sequence that was used during encryption. A chaotic pseudo-random number generator (PRNG) is initialized with identical parameters to reproduce the exact keystream. The encrypted image is then combined with this regenerated chaotic sequence using a bitwise XOR

operation. Since XOR is a reversible operation, applying the same keystream to the encrypted data successfully retrieves the original image.

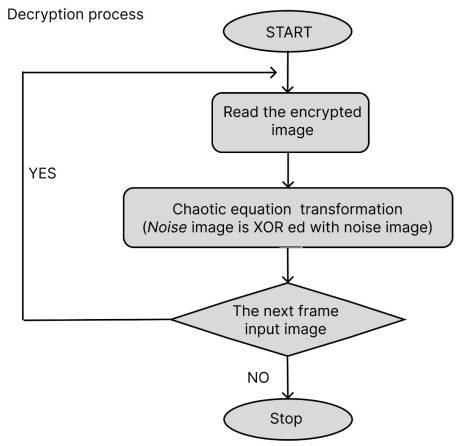


Figure 3.2: Flowchart of Decryption Process

The top-level block diagram of the system is shown in Fig 3.3 . It illustrates the flow of pixel data from input to encrypted output through the encryption subsystem.

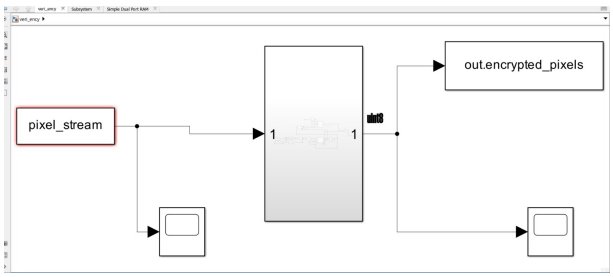


Figure 3.3: Block Diagram of Top Module

The architecture is designed to ensure continuous data processing and is suitable for real-time applications such as secure image transmission in surveillance systems.

3.2 ENCRYPTION (CORE) SUBSYSTEM

The encryption core is the fundamental unit responsible for transforming input pixel data into encrypted output using a chaotic keystream. The subsystem was initially designed in Simulink using HDL blocks and later synthesized into FPGA logic. The subsystem consists of:

- Chaotic sequence generator
- Memory interface (RAM)
- XOR-based encryption unit

- FIFO buffer for data flow control
- Control logic (counters and enable signals)

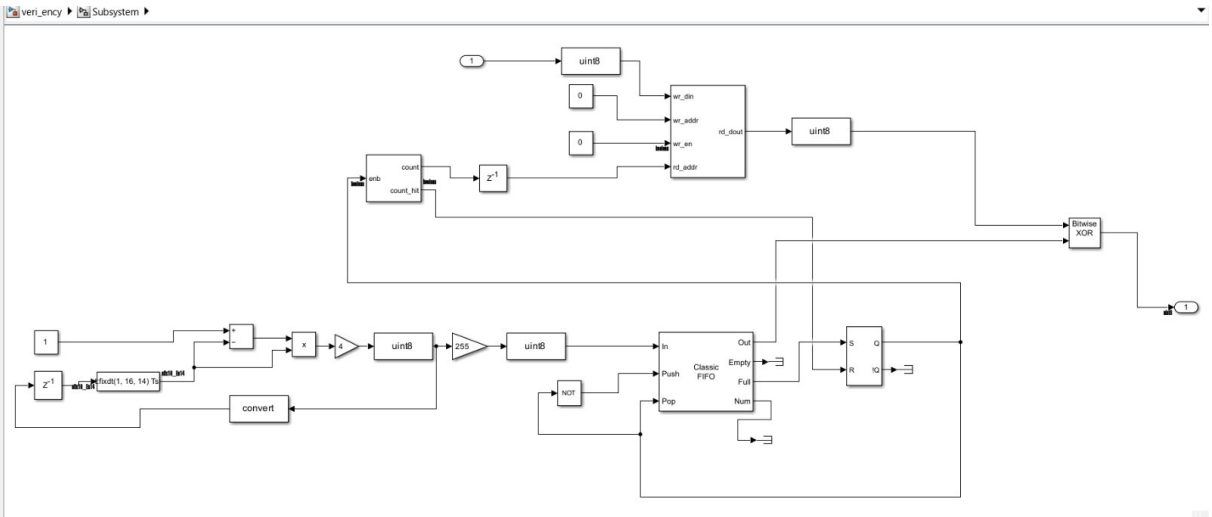


Figure 3.4: Block Diagram of Encryption Subsystem

3.2.1 Chaotic Map Implementation (Logistic Core)

The system uses a logistic map to generate pseudo-random sequences, which act as the encryption key stream. The mathematical model of the logistic map is given by:

$$x_{n+1} = r \cdot x_n \cdot (1 - x_n) \tag{3.2}$$

In the hardware implementation:

- A delay element (z^{-1}) stores the previous value of the sequence
- Arithmetic blocks perform multiplication and subtraction operations
- 32-bit Fixed-point representation is used to ensure efficient FPGA realization

The generated chaotic values are scaled and converted into 8-bit format to match the pixel data width.

3.2.2 Nonlinear Coupling Logic

Nonlinearity is introduced through feedback and arithmetic operations within the chaotic generator. The recursive nature of the logistic map, along with multiplication and scaling operations, enhances the randomness of the generated sequence.
From Fig.3.4., it can be observed that:

- Previous outputs are fed back into the system
- Arithmetic blocks introduce nonlinear transformations
- Data type conversion blocks ensure compatibility between stages

This significantly improves the unpredictability of the keystream and strengthens the encryption process.

3.2.3 Keystream Extraction Logic (XOR-based Diffusion)

The final encryption operation is performed using a bitwise XOR between the pixel data and the generated chaotic sequence:

$$C(i) = P(i) \oplus K(i) \tag{3.3}$$

In the subsystem:

- Pixel data is fetched from RAM
- Chaotic sequence is generated simultaneously
- Both inputs are provided to a Bitwise XOR block

This operation ensures strong diffusion, where small changes in input or key result in significant variations in output.

3.3 MEMORY MANAGEMENT

Efficient memory management plays a crucial role in the overall performance of FPGA-based image processing systems. In the proposed design, memory is used to store and retrieve pixel data in a structured manner, ensuring smooth data flow between different processing stages. Although the current implementation is not fully real-time, the memory architecture is designed to support sequential processing efficiently and can be extended for real-time applications.

3.3.1 Dual-Port RAM Usage

As shown in Fig.3.4, a dual-port RAM is employed to facilitate simultaneous read and write operations. This allows the system to store incoming pixel values while concurrently

supplying previously stored data for encryption or decryption. The use of dual-port memory eliminates access conflicts and improves throughput.
In this design, the memory operates as follows:

- One port is dedicated to writing pixel data into memory.
- The second port is used to read pixel values for further processing.

A counter-based addressing mechanism is used to generate sequential memory addresses, ensuring orderly access to pixel data. This structured approach simplifies memory control and enhances the efficiency of data handling within the system.

3.3.2 Handling 2500-Pixel Data Arrays

The system is designed to process a grayscale image of size 50×50, resulting in a total of 2500 pixels. These pixel values are stored in memory and accessed sequentially during processing. The use of a counter enables systematic traversal of the entire image without skipping or overwriting data.
The processing mechanism can be summarized as follows:

- A counter generates addresses ranging from 0 to 2499.
- Each pixel is accessed in a sequential manner.
- A delay element (z^{-1}) is introduced to maintain synchronization between memory access and processing stages.

This design ensures that each pixel is processed accurately while maintaining proper timing alignment between different modules. The sequential access pattern reduces control complexity and prevents data hazards, making the system reliable and scalable for larger image sizes in future implementations.

3.4 FIFO BASED BUFFERING

In addition to memory storage using RAM, a FIFO (First-In-First-Out) buffer is incorporated in the design to regulate the flow of data between different processing stages. Since the system consists of multiple modules such as memory access, chaotic sequence generation, and encryption logic, each operating at slightly different rates, direct data transfer may lead to synchronization issues. The FIFO buffer acts as an intermediate storage element that ensures smooth and controlled data movement across these stages.

The FIFO operates on the principle that the first data written into the buffer is the first data to be read out. This property is particularly useful in maintaining the correct order of pixel data during processing. In the proposed system, pixel values or intermediate encrypted data are temporarily stored in the FIFO before being forwarded to the next stage, such as XOR-based encryption or UART transmission.
The operation of the FIFO is controlled using standard control signals:

- **Push (Write Enable):** Inserts data into the FIFO when valid input is available.
- **Pop (Read Enable):** Removes data from the FIFO for further processing.
- **Full Signal:** Indicates that the FIFO cannot accept new data, preventing overflow.
- **Empty Signal:** Indicates that no data is available for reading, preventing underflow.

By incorporating FIFO buffering, the design achieves better synchronization between modules and avoids data loss due to mismatched processing speeds. It also helps in decoupling the producer (data generation stage) and consumer (encryption or transmission stage), thereby improving overall system stability.

3.5 FINITE STATE MACHINE DESIGN

The system operation is controlled using a Finite State Machine (FSM), which ensures proper sequencing of encryption and decryption processes.

3.5.1 Phase 1: Encryption State

- Pixel data is read from memory
- Chaotic keystream is generated
- XOR operation produces encrypted output
- Data is stored or transmitted via UART

3.5.2 Phase 2: Decryption State

- Encrypted data is fed back into the FPGA
- Identical keystream is generated using same initial conditions
- XOR operation reconstructs original pixel values

- Decrypted data is transmitted via UART

3.6 COMMUNICATION INTERFACE

3.6.1 UART Transmitter Design

The encrypted pixel data is transmitted from the FPGA using a UART transmitter module. The UART operates based on asynchronous serial communication and includes:

- Start bit (logic low)
- 8-bit data transmission
- Stop bit (logic high)

The baud rate is configured based on the system clock (100 MHz on Nexys DDR board). A baud rate generator is used to derive the required transmission speed.
The transmitted data is received on a host system using Python, where the encrypted image can be reconstructed. Similarly, decrypted data from the FPGA is also transmitted via UART for verification.

3.7 RTL SCHEMATIC AND HARDWARE INTEGRATION

The RTL schematic generated in Vivado provides a detailed view of the hardware implementation. The design integrates multiple FPGA primitives and I/O components.

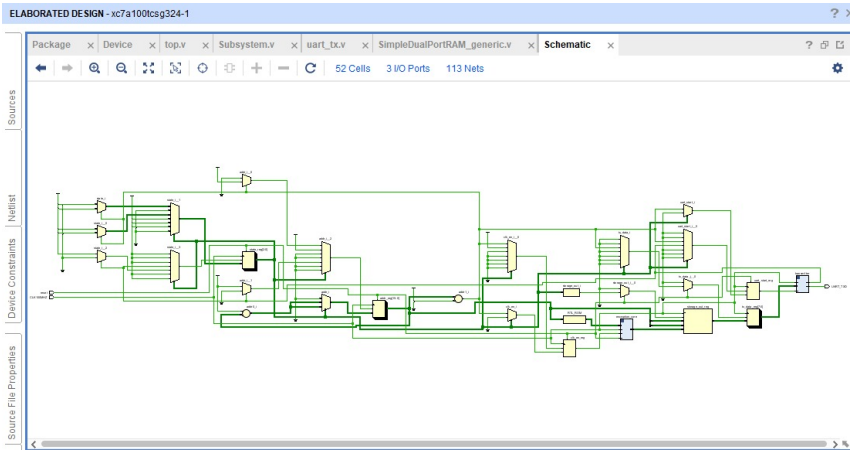


Figure 3.5: RTL Schematic of Entire System Generated in Vivado

The RTL schematic shown in Fig.3.5 represents the elaborated hardware architecture of the encryption and decryption subsystem after synthesis in Vivado. This schematic provides a structural view of how the design is mapped onto FPGA primitives and how data flows through different functional blocks.

The subsystem consists of multiple interconnected modules including control logic, memory units, chaotic sequence generator, arithmetic blocks, FIFO buffer, and UART interface.

3.7.1 Input and Control Signal Routing

The leftmost section of the schematic shows input signal distribution and control logic. The system clock is distributed to all sequential elements through buffered clock paths to ensure synchronized operation. Control signals such as enable, reset, and data-valid signals are propagated through combinational logic blocks and multiplexers.

Several LUT-based logic elements are used to generate enable conditions and gating signals required for proper sequencing of operations. These control paths ensure that read, write, encryption, and transmission stages operate in a coordinated manner.

3.7.2 Memory Interface and Address Generation

The design incorporates a dual-port RAM module for storing pixel data. The RAM is interfaced using the following signals:

- Write data input (wr_din)
- Write address (wr_addr)
- Write enable (wr_en)
- Read address (rd_addr)
- Read data output (rd_dout)

A counter-based addressing mechanism is implemented using sequential logic elements. The counter increments with each clock cycle and generates addresses for accessing pixel locations in memory. This ensures sequential processing of image data.

3.7.3 Chaotic Sequence Generator

The central portion of the schematic implements the chaotic PRNG using arithmetic blocks. The design realizes the logistic map equation through a combination of multipliers, adders, and feedback registers.

- Flip-flops store previous state values
- Multipliers and subtractors implement nonlinear operations
- Scaling factors are applied using constant multipliers

The recursive feedback structure ensures that each output depends on the previous state, resulting in a highly unpredictable sequence. The output of this block forms the keystream used for encryption and decryption.

3.7.4 Data Conversion and Scaling

The chaotic sequence generated is processed through conversion blocks to match the required data width. Scaling operations are used to map the generated values into an 8-bit range suitable for pixel-level operations.
This stage ensures compatibility between the chaotic generator output and pixel data stored in memory.

3.7.5 XOR-Based Encryption and Decryption

The core encryption operation is implemented using bitwise XOR logic. The pixel data retrieved from RAM is combined with the chaotic keystream:

$$C(i) = P(i) \oplus K(i) \tag{3.4}$$

Since XOR is a reversible operation, the same logic is reused for decryption. When the encrypted data is XORed again with the same keystream, the original pixel values are recovered.
The XOR logic is implemented using LUT-based combinational elements in the FPGA fabric.

3.7.6 FIFO Buffer and Data Flow Control

A FIFO (First-In-First-Out) buffer is included in the design to regulate data flow between processing stages. The FIFO helps in decoupling the processing speed of different modules and ensures smooth data transfer.
The FIFO is controlled using:

- Push (write enable)

- Pop (read enable)
- Empty and Full status signals

These signals are generated through control logic and ensure that data is neither lost nor overwritten.

3.7.7 UART Transmission Interface

The rightmost section of the schematic represents the UART transmission module. The processed pixel data (encrypted or decrypted) is serialized and transmitted through the UART TX line.
The UART module includes:

- Baud rate generator
- Shift register for serialization
- Start and stop bit control logic

This enables communication between the FPGA and external systems such as a computer, where the transmitted data is received and reconstructed using Python.

3.7.8 Overall Data Flow

The complete data flow in the subsystem can be summarized as follows:

$$\text{Pixel Data} \rightarrow \text{RAM} \rightarrow \text{Chaotic Generator} \rightarrow \text{XOR} \rightarrow \text{FIFO} \rightarrow \text{UART}$$

For decryption, the encrypted data is reprocessed through the same XOR logic using an identical keystream, thereby reconstructing the original image.

3.7.9 Conclusion

The RTL schematic confirms that the design is fully synthesizable and efficiently mapped onto FPGA resources. The modular architecture, combined with proper control logic and buffering mechanisms, ensures reliable and real-time operation of the image encryption and decryption system.

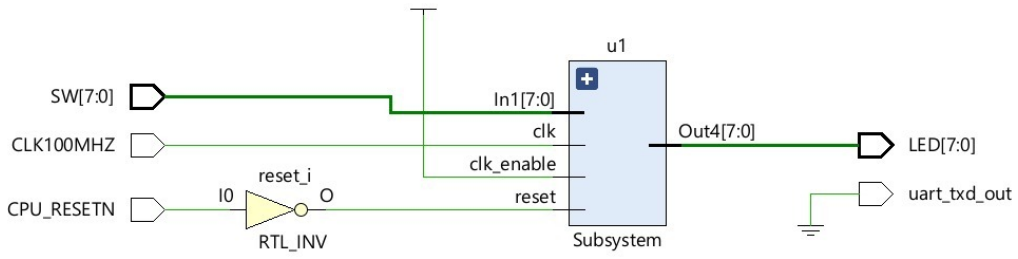


Figure 3.6: RTL Schematic of Top Module Generated in Vivado

The RTL schematic represents the top-level hardware integration of the system on FPGA. It shows the interconnection between external inputs such as switches (SW[7:0]), clock (CLK100MHZ), and reset signal with the internal processing module.

The reset signal is passed through an inverter to convert the active-low reset input into an active-high signal required by the subsystem. The main processing unit, labeled as Subsystem (u1), performs encryption and decryption operations.

The subsystem receives pixel input, clock, and reset signals, and produces processed output which is routed to LED outputs for visualization and to the UART transmission line for communication with an external system. This schematic confirms correct integration of FPGA I/O with the designed encryption architecture.

The subsystem internally contains the encryption and decryption logic, memory blocks, and control units. The UART transmission block is connected to the output port, enabling communication with external systems.

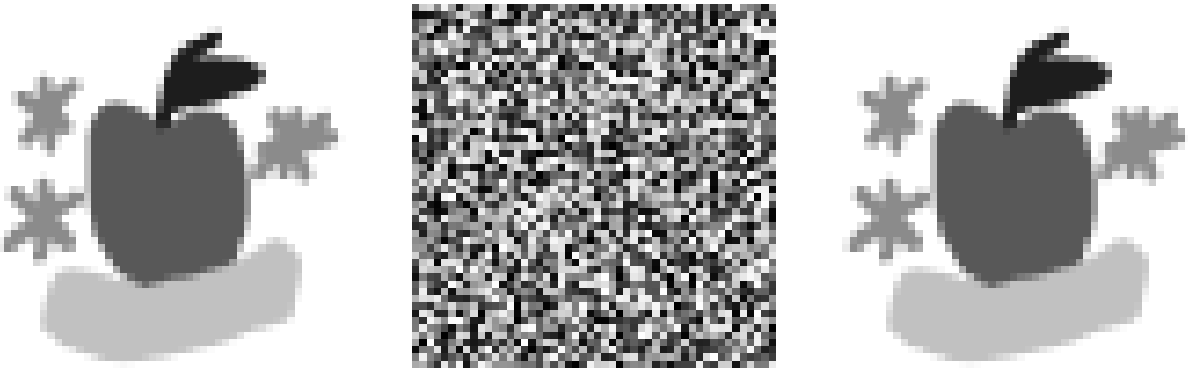
This schematic confirms that the design is fully synthesizable and successfully mapped onto FPGA hardware resources.

3.8. RESULTS

The results of the proposed image encryption and decryption system are shown in Fig.3.7. The original image represents the input grayscale data, while the encrypted image is obtained by applying a chaotic PRNG-based XOR operation.

The encrypted image appears noise-like and does not reveal any visible information about the original content, demonstrating effective encryption. During decryption, the same chaotic sequence is regenerated and XORed with the encrypted image to recover the original data.

The decrypted image closely matches the original image, confirming the correctness and reversibility of the proposed method. This validates that the system achieves secure encryption while preserving data integrity.



(a) Original Image (b) Encrypted image (c) Decrypted image

Figure 3.7: Original, Encrypted and Decrypted Image

3.8.CONCLUSION OF PROPOSED SYSTEM

The proposed FPGA-based image encryption system effectively combines chaotic encryption techniques with efficient hardware implementation. The integration of RAM, FIFO buffering, and FSM control enables real-time processing with high reliability. The modular design also allows scalability for higher resolution images and advanced communication interfaces.

CHAPTER 4
HARDWARE IMPLEMENTATION

4.1 INTRODUCTION

The hardware implementation of the proposed image encryption and decryption system is carried out using a Field Programmable Gate Array (FPGA). Compared to software-based implementations, hardware realization provides high-speed processing, parallel execution, and deterministic performance, making it suitable for real-time secure image transmission applications.

In this project, the system is designed using Verilog Hardware Description Language (HDL) and synthesized using Xilinx Vivado targeting an Artix-7 FPGA. The implementation focuses on efficient realization of chaotic sequence generation, pixel-wise encryption, memory handling, and UART-based communication.

4.2 TARGET HARDWARE PLATFORM

The design is implemented on a Nexys FPGA development board based on the Artix-7 architecture. The platform is selected due to its high logic density, availability of Block RAM (BRAM), clock management resources, and built-in UART support.

The FPGA operates at a system clock frequency of 100 MHz, enabling fast execution of chaotic iterations and pixel processing.

4.3 DESIGN FLOW

The hardware implementation follows a structured design methodology:

- **Algorithm Modeling:** The chaotic encryption algorithm is verified using MATLAB/Simulink with HDL-compatible blocks.
- **HDL Coding:** The system is implemented using Verilog HDL with a modular approach.

- **Simulation:** Functional verification is performed using testbenches.
- **Synthesis:** The design is synthesized using Xilinx Vivado.
- **Hardware Testing:** The system is tested on FPGA using UART communication.

4.4 HARDWARE ARCHITECTURE

The FPGA implementation consists of multiple modules forming a complete encryption system.

4.4.1 Chaotic Sequence Generator

The chaotic generator is implemented using a two-dimensional logistic map. Fixed-point arithmetic (Q2.30 format) is used to efficiently realize the nonlinear equations in hardware.
Two state variables are iteratively updated using nonlinear transformations and feedback. Fixed-point implementation reduces hardware complexity while maintaining precision.

4.4.2 Encryption Module

Encryption is performed using a bitwise XOR operation between the input pixel and the generated chaotic keystream:

$$C(i) = P(i) \oplus K(i) \tag{4.5}$$

where $P(i)$ is the input pixel, $K(i)$ is the keystream, and $C(i)$ is the encrypted output.

4.4.3 Memory Implementation

- Dual-port RAM for simultaneous read and write operations
- Preloaded memory using initialization files
- Separate input and output buffers

This improves throughput and supports real-time processing.

4.4.4 FIFO Buffer

A synchronous FIFO (First-In-First-Out) buffer is used to manage data flow between modules. It helps in:

- Handling speed mismatches
- Preventing data loss
- Maintaining correct data sequence

The FIFO uses read and write pointers along with status signals such as Full and Empty.

4.4.5 Control Unit(FSM)

A Finite State Machine (FSM) is used to control system operation. The main states include:

- Initialization
- Encryption
- Data storage
- Transmission

The FSM ensures proper synchronization between different modules.

4.4.6 UART Communication Module

The UART transmitter is used to send encrypted data to an external system. It operates using asynchronous communication with:

- 1 Start bit
- 8 Data bits
- 1 Stop bit

A baud rate generator is used to match standard communication speeds such as 115200 baud.

4.5 TOP LEVEL INTEGRATION

All modules are integrated in a top-level design that performs:

1. Reading pixel data from memory

2. Encrypting pixels using chaotic core
3. Storing encrypted data
4. Transmitting data via UART

The system processes the entire image sequentially while maintaining synchronization between modules.

4.6 RTL DESIGN AND RESOURCE UTILIZATION

The RTL schematic generated using Vivado confirms that the design is fully synthesizable. The implementation efficiently utilizes FPGA resources such as:

- Lookup Tables (LUTs)
- Flip-Flops
- Block RAM (BRAM)
- DSP slices

The use of fixed-point arithmetic reduces hardware usage while maintaining high performance.

4.7 HARDWARE TESTING AND VALIDATION

The system is tested on FPGA hardware by:

- Loading input image data using memory files
- Observing encrypted output via UART
- Capturing data on a PC interface
- Verifying correctness through decryption

The results confirm successful real-time encryption and accurate reconstruction during decryption.

4.8 ENTROPY ANALYSIS

Entropy is used to measure the randomness or unpredictability of pixel intensity values in an image. It is calculated using Shannon's entropy formula:

$$H = -\sum_{i=0}^{255} p(i) \log_2 p(i) \tag{4.6}$$

where $p(i)$ represents the probability of occurrence of the pixel intensity value i. The probability is obtained from the normalized histogram of the image and is given by:

$$p(i) = \frac{n_i}{N} \tag{4.7}$$

Here, n_i is the number of pixels with intensity i, and N is the total number of pixels in the image.

The entropy values for the original, encrypted, and decrypted images were computed using the above equations. The results are illustrated in Fig. 4.1.

The original image has an entropy of approximately 2.83, indicating a high level of redundancy and structured information. In contrast, the encrypted image achieves an entropy of 7.93, which is close to the ideal value of 8 for an 8-bit image, demonstrating a high degree of randomness and effective encryption.

The decrypted image restores the entropy back to approximately 2.83, confirming that the decryption process accurately reconstructs the original image without any loss of information. This validates that the proposed chaotic encryption scheme provides strong statistical security while ensuring correct data recovery.

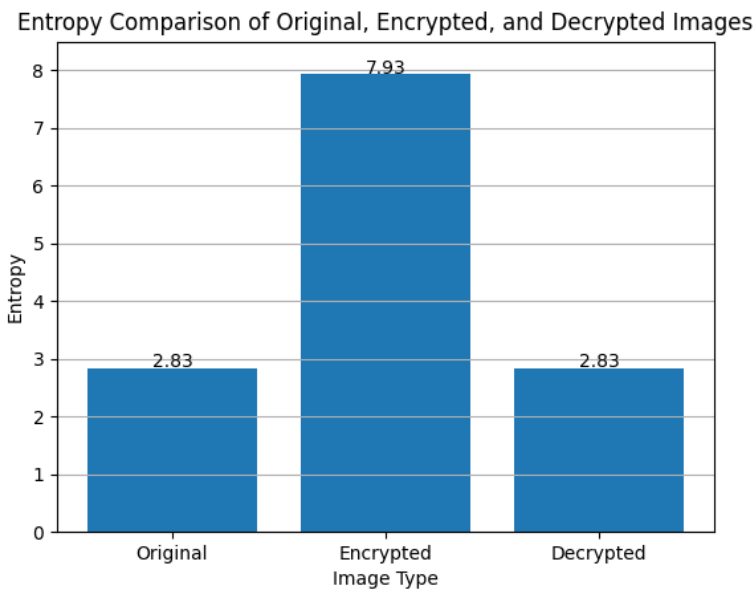


Figure 4.1: Entropy Comparison of Original, Encrypted, and Decrypted Images

CHAPTER 5
HDL IMPLEMENTATION

5.1 Subsystem.v

```
`timescale 1ns/1ps

module Subsystem
(
    input clk,
    input reset,
    input clk_enable,
    input [7:0] In1,
    output ce_out,
    output [7:0] Out4
);

assign ce_out = clk_enable;

////////////////////////////////////////////////////////////////
// 32-bit High Precision 2D Chaotic System (Q2.30)
////////////////////////////////////////////////////////////////

// 1.0 in Q2.30
parameter signed [31:0] ONE = 32'sd1073741824;

// r = 3.999 in Q2.30
parameter signed [31:0] R = 32'sd4292870140 >> 2;
```

```
24
25 // Seeds (MUST be same in encryption & decryption)
26 parameter signed [31:0] SEED1 = 32'sd132120576;
27 parameter signed [31:0] SEED2 = 32'sd268435456;
28
29 ////////////////////////////////////////////////////////////////
30 // State Registers
31 ////////////////////////////////////////////////////////////////
32
33 reg signed [31:0] x;
34 reg signed [31:0] y;
35
36 ////////////////////////////////////////////////////////////////
37 // Intermediate Signals
38 ////////////////////////////////////////////////////////////////
39
40 wire signed [31:0] one_minus_x;
41 wire signed [31:0] one_minus_y;
42
43 wire signed [63:0] mult_x;
44 wire signed [63:0] mult_y;
45
46 wire signed [63:0] rx_full;
47 wire signed [63:0] ry_full;
48
49 wire signed [31:0] x_next;
50 wire signed [31:0] y_next;
51
52 ////////////////////////////////////////////////////////////////
53 // Logistic Core
54 ////////////////////////////////////////////////////////////////
55
56 assign one_minus_x = ONE - x;
57 assign one_minus_y = ONE - y;
58
```

```
assign mult_x = x * one_minus_x;   // Q4.60
assign mult_y = y * one_minus_y;

assign rx_full = mult_x * R;       // multiply by r
assign ry_full = mult_y * R;

// Back to Q2.30
wire signed [31:0] rx_scaled = rx_full[61:30];
wire signed [31:0] ry_scaled = ry_full[61:30];

////////////////////////////////////////////////////////////
// Strong Nonlinear Coupling
////////////////////////////////////////////////////////////

wire signed [63:0] xy_mult = x * y;

assign x_next = rx_scaled ^ (y >>> 5) ^ xy_mult[45:14];
assign y_next = ry_scaled ^ (x >>> 5) ^ xy_mult[50:19];

////////////////////////////////////////////////////////////
// State Update (Deterministic)
////////////////////////////////////////////////////////////

always @(posedge clk or posedge reset)
begin
    if (reset) begin
        x <= SEED1;
        y <= SEED2;
    end
    else if (clk_enable) begin
        x <= x_next;
        y <= y_next;
    end
end

```

```
////////////////////////////////////////////////////////////
// Strong Keystream Extraction
////////////////////////////////////////////////////////////

wire [7:0] keystream;

assign keystream =
        x[31:24] ^
        x[23:16] ^
        y[29:22] ^
        {x[5:0], y[7:6]};

////////////////////////////////////////////////////////////
// XOR Encryption (Reversible)
////////////////////////////////////////////////////////////

assign Out4 = In1 ^ keystream;

endmodule
```

Listing 5.1: Subsystem Module

Subsystem.v Verilog module acts as a digital lock and key generator for image data. It uses a mathematical ”chaos” engine to create a constantly changing 8-bit secret key (the keystream) for every single pixel that enters the system. By using high-precision 32-bit math, the code calculates two complex variables, x and y, which jump around in a way that looks random but is actually a perfect sequence based on the initial ”seeds.” When a pixel arrives at the In1 port, the module extracts a piece of this chaotic sequence and mixes it with the pixel using an XOR operation. This transforms the original image into scrambled noise (Out4) that can only be unscrambled if someone has the exact same starting seeds to recreate the identical ”key” sequence.

5.2 HDL_FIFO

```
// ------------------------------------------------------------
//
```

```
// File Name: hdl_prj\hdlsrc\veri_ency\HDL_FIFO.v
// Created: 2026-02-12 21:36:13
//
// Generated by MATLAB 25.2, HDL Coder 25.2, and Simulink 25.2
//
// -------------------------------------------------------------


// -------------------------------------------------------------
//
// Module: HDL_FIFO
// Source Path: veri_ency/Subsystem/HDL_FIFO
// Hierarchy Level: 1
// Model version: 1.4
//
// -------------------------------------------------------------

`timescale 1 ns / 1 ns

module HDL_FIFO
          (clk,
           reset,
           enb,
           In,
           Push,
           Pop,
           Out,
           Full);


  input   clk;
  input   reset;
  input   enb;
  input   [7:0] In;  // uint8
  input   Push;
```

```
38  input   Pop;
39  output  [7:0] Out;  // uint8
40  output  Full;  // ufix1
41
42
43  reg [11:0] fifo_front_indx;  // ufix12
44  reg [11:0] fifo_front_dir;  // ufix12
45  reg [11:0] fifo_back_indx;  // ufix12
46  reg [11:0] fifo_back_dir;  // ufix12
47  reg [11:0] fifo_sample_count;  // ufix12
48  wire [11:0] fifo_front_indx_next;  // ufix12
49  wire [11:0] fifo_front_dir_next;  // ufix12
50  wire [11:0] fifo_back_indx_next;  // ufix12
51  wire [11:0] fifo_back_dir_next;  // ufix12
52  wire [11:0] fifo_sample_count_next;  // ufix12
53  wire fifo_out3;
54  wire fifo_out4;
55  wire fifo_write_enable;
56  wire fifo_read_enable;
57  wire [11:0] fifo_front_indx_temp;  // ufix12
58  wire [11:0] fifo_back_indx_temp;  // ufix12
59  wire [11:0] w_waddr;  // ufix12
60  wire w_we;  // ufix1
61  wire [11:0] w_raddr;  // ufix12
62  wire Empty;  // ufix1
63  wire [11:0] Num;  // ufix12
64  wire w_cz;
65  wire w_const;  // ufix1
66  wire w_mux1;  // ufix1
67  reg  w_d1;  // ufix1
68  wire [7:0] w_waddr_1;  // uint8
69  reg [7:0] w_d2;  // uint8
70  wire [7:0] w_out;  // uint8
71
72
```

```
// FIFO logic controller
// us2: Upsample by 1, Sample offset 0
// us3: Upsample by 1, Sample offset 0
always @(posedge clk or posedge reset)
  begin : fifo_process
    if (reset == 1'b1) begin
      fifo_front_indx <= 12'b000000000000;
      fifo_front_dir <= 12'b000000000001;
      fifo_back_indx <= 12'b000000000000;
      fifo_back_dir <= 12'b000000000001;
      fifo_sample_count <= 12'b000000000000;
    end
    else begin
      if (enb) begin
        fifo_front_indx <= fifo_front_indx_next;
        fifo_front_dir <= fifo_front_dir_next;
        fifo_back_indx <= fifo_back_indx_next;
        fifo_back_dir <= fifo_back_dir_next;
        fifo_sample_count <= fifo_sample_count_next;
      end
    end
  end

assign fifo_out4 = fifo_sample_count == 12'b100111000100;
assign fifo_out3 = fifo_sample_count == 12'b000000000000;
assign fifo_write_enable = Push && (Pop || ( ! fifo_out4));
assign fifo_read_enable = Pop && ( ! fifo_out3);
assign fifo_front_indx_temp = (fifo_read_enable ? fifo_front_indx +
    fifo_front_dir :
            fifo_front_indx);
assign fifo_front_dir_next = (fifo_front_indx_temp == 12'
   b100111000011 ? 12'b011000111101 :
            12'b000000000001);
assign fifo_back_indx_temp = (fifo_write_enable ? fifo_back_indx +
   fifo_back_dir :
```

```
            fifo_back_indx);
assign fifo_back_dir_next = (fifo_back_indx_temp == 12'b100111000011 ? 12'b011000111101 :
            12'b000000000001);
assign fifo_sample_count_next = (fifo_write_enable && ( ! fifo_read_enable) ? fifo_sample_count + 12'b000000000001 :
            (( ! fifo_write_enable) && fifo_read_enable ? fifo_sample_count + 12'b111111111111 :
            fifo_sample_count));
assign w_waddr = fifo_back_indx;
assign w_we = fifo_write_enable;
assign w_raddr = fifo_front_indx;
assign Empty = fifo_out3;
assign Full = fifo_out4;
assign Num = fifo_sample_count;
assign fifo_front_indx_next = fifo_front_indx_temp;
assign fifo_back_indx_next = fifo_back_indx_temp;

assign w_cz = Num > 12'b000000000000;

assign w_const = 1'b0;

assign w_mux1 = (w_cz == 1'b0 ? w_const :
            Pop);

always @(posedge clk or posedge reset)
  begin : f_d1_process
    if (reset == 1'b1) begin
      w_d1 <= 1'b0;
    end
    else begin
      if (enb) begin
        w_d1 <= w_mux1;
      end
    end
```

```
137     end
138
139   // us1: Upsample by 1, Sample offset 0
140   SimpleDualPortRAM_generic #(.AddrWidth(12),
141                               .DataWidth(8)
142                               )
143                            u_HDL_FIFO_ram_generic (.clk(clk),
144                                                    .enb(enb),
145                                                    .wr_din(In),
146                                                    .wr_addr(w_waddr)
                                                         ,
147                                                    .wr_en(w_we),   //
                                                         ufix1
148                                                    .rd_addr(w_raddr)
                                                         ,
149                                                    .dout(w_waddr_1)
150                                                    );
151
152   always @(posedge clk or posedge reset)
153     begin : f_d2_process
154       if (reset == 1'b1) begin
155         w_d2 <= 8'b00000000;
156       end
157       else begin
158         if (enb && w_d1) begin
159           w_d2 <= w_waddr_1;
160         end
161       end
162     end
163
164   assign w_out = (w_d1 == 1'b0 ? w_d2 :
165               w_waddr_1);
166
167   assign Out = w_out;
168
```

```
endmodule  // HDL_FIFO
```

Listing 5.2: HDL_FIFO Module

The system incorporates an 8-bit Synchronous FIFO (First-In, First-Out) Buffer used to manage data flow and timing between different stages of the encryption architecture. It utilizes a Simple Dual Port RAM for data storage, coordinated by a logic controller that tracks read and write addresses using 12-bit pointers (`fifo_front_indx` and `fifo_back_indx`). The module automatically handles critical status signals like *Full* and *Empty* by monitoring the sample count, ensuring that data is only ”pushed” when there is space and ”popped” when data is available. This acts as a vital coordination layer, preventing data loss and smoothing out communication between the high-speed chaotic core and slower external interfaces like the UART.

5.3 SimpleDualPortRAM_generic.v

```
// ------------------------------------------------------------
//
// File Name: hdl_prj\hdlsrc\veri_ency\SimpleDualPortRAM_generic.v
// Created: 2026-02-12 21:36:13
//
// Generated by MATLAB 25.2, HDL Coder 25.2, and Simulink 25.2
//
// ------------------------------------------------------------


// ------------------------------------------------------------
//
// Module: SimpleDualPortRAM_generic
// Source Path: veri_ency/Subsystem/HDL_FIFO/
   SimpleDualPortRAM_generic
// Hierarchy Level: 1
// Model version: 1.4
//
// ------------------------------------------------------------

```

```
20 `timescale 1 ns / 1 ns
21
22 module SimpleDualPortRAM_generic
23          (clk,
24           enb,
25           wr_din,
26           wr_addr,
27           wr_en,
28           rd_addr,
29           dout);
30
31   parameter integer AddrWidth = 1;
32   parameter integer DataWidth = 1;
33
34   input   clk;
35   input   enb;
36   input   [DataWidth - 1:0] wr_din;  // parameterized width
37   input   [AddrWidth - 1:0] wr_addr;  // parameterized width
38   input   wr_en;  // ufix1
39   input   [AddrWidth - 1:0] rd_addr;  // parameterized width
40   output  [DataWidth - 1:0] dout;  // parameterized width
41
42
43   reg  [DataWidth - 1:0] ram [2**AddrWidth - 1:0];
44   reg  [DataWidth - 1:0] data_int;
45   integer i;
46
47 initial begin
48   $readmemh("apple_pixels_vivado.mem", ram);
49   data_int = 0;
50 end
51
52
53
54   always @(posedge clk)
```

```
    begin : SimpleDualPortRAM_generic_process
      if (enb == 1'b1) begin
        if (wr_en == 1'b1) begin
          ram[wr_addr] <= wr_din;
        end
        data_int <= ram[rd_addr];
      end
    end

  assign dout = data_int;

endmodule  // SimpleDualPortRAM_generic
```

Listing 5.3: `SimpleDualPortRAM_generic` Module

This Verilog module implements a Parameterized Simple Dual-Port RAM, which serves as the primary data storage unit for the image encryption system. It is designed for efficiency on FPGA hardware, allowing the system to simultaneously write new pixel data to one memory location (`wr_addr`) while reading existing data from another (`rd_addr`) within the same clock cycle. A key feature of this implementation is the use of the `$readmemh` system task, which pre-loads the memory with the `apple_pixels_vivado.mem` file during hardware initialization. By using parameters for *AddrWidth* and *DataWidth*, the module remains highly flexible, scaling to accommodate different image resolutions or bit depths without requiring modifications to the core architectural logic.

5.4 veri_ency.v

```
module veri_ency
          (clk,
           reset,
           clk_enable,
           In1,
           ce_out,
           Out1);

  input        clk;
```

```
  input        reset;
  input        clk_enable;
  input  [7:0] In1;
  output       ce_out;
  output [7:0] Out1;

  wire [7:0] Subsystem_Out4;

  Subsystem u_Subsystem (
    .clk(clk),
    .reset(reset),
    .clk_enable(clk_enable),
    .In1(In1),              //     FIXED
    .ce_out(),
    .Out4(Subsystem_Out4)
  );

  assign Out1   = Subsystem_Out4;
  assign ce_out = clk_enable;

endmodule
```

Listing 5.4: veri_ency Module

It serves as the Top-Level Wrapper for the image encryption project. Its primary role is to act as the external interface for the FPGA, connecting the physical input/output pins to the internal logic of the `Subsystem` module. By instantiating the `u_Subsystem` component, it directs the incoming 8-bit pixel data (`In1`) into the chaotic encryption core and routes the resulting processed data to the final output (`Out1`). This structural approach ensures a clean hierarchy, where global signals like the system clock (`clk`) and hardware reset (`reset`) are cleanly distributed, providing a stable environment for the high-speed image encryption process to occur.

5.5 S_R_Flip_Flop.v

```
// ------------------------------------------------------------
```

```
//
// File Name: hdl_prj\hdlsrc\veri_ency\S_R_Flip_Flop.v
// Created: 2026-02-12 21:36:13
//
// Generated by MATLAB 25.2, HDL Coder 25.2, and Simulink 25.2
//
// -------------------------------------------------------------


// -------------------------------------------------------------
//
// Module: S_R_Flip_Flop
// Source Path: veri_ency/Subsystem/S-R Flip Flop
// Hierarchy Level: 1
// Model version: 1.4
//
// -------------------------------------------------------------

`timescale 1 ns / 1 ns

module S_R_Flip_Flop
          (clk,
           reset,
           enb,
           in1,
           in2,
           out1);


  input   clk;
  input   reset;
  input   enb;
  input   in1;
  input   in2;
  output  out1;
```

```
37
38
39  reg  Previous_Q;
40  wire [2:0] InpCombined;  // ufix3
41  wire [2:0] S_R_Flip_FlopDirect_Lookup_Table_1 [0:7];  // ufix3 [8]
42  wire [2:0] OutCombined;  // ufix3
43
44
45  always @(posedge clk or posedge reset)
46    begin : S_R_Flip_Flop_memory_process
47      if (reset == 1'b1) begin
48        Previous_Q <= 1'b0;
49      end
50      else begin
51        if (enb) begin
52          Previous_Q <= out1;
53        end
54      end
55    end
56
57  assign InpCombined = {in1, in2, Previous_Q};
58
59  assign S_R_Flip_FlopDirect_Lookup_Table_1[0] = 3'b001;
60  assign S_R_Flip_FlopDirect_Lookup_Table_1[1] = 3'b010;
61  assign S_R_Flip_FlopDirect_Lookup_Table_1[2] = 3'b001;
62  assign S_R_Flip_FlopDirect_Lookup_Table_1[3] = 3'b001;
63  assign S_R_Flip_FlopDirect_Lookup_Table_1[4] = 3'b010;
64  assign S_R_Flip_FlopDirect_Lookup_Table_1[5] = 3'b010;
65  assign S_R_Flip_FlopDirect_Lookup_Table_1[6] = 3'b000;
66  assign S_R_Flip_FlopDirect_Lookup_Table_1[7] = 3'b000;
67  assign OutCombined = S_R_Flip_FlopDirect_Lookup_Table_1[InpCombined
      ];
68
69  assign out1 = OutCombined[1];
70
```

```
endmodule  // S_R_Flip_Flop
```

Listing 5.5: S_R_Flip_Flop Module

It implements a Set-Reset (S-R) Flip-Flop using a Lookup Table (LUT) approach, which is a common optimization for FPGA synthesis. Unlike a traditional gate-level implementation, this module combines the current inputs (in1, in2) and the feedback state (Previous_Q) into a 3-bit address to instantly retrieve the next state from a pre-defined table. It serves as a fundamental sequential logic element within the design, providing a stable 1-bit memory latch that can be "Set" or "Reset" to control state transitions or enable signals across different clock cycles.

5.6 uart_tx.v

```
`timescale 1ns/1ps

module uart_tx(
    input clk,
    input reset,
    input start,
    input [7:0] data,
    output reg tx,
    output reg busy
);

parameter BAUD_DIV = 868; // 115200 baud @ 100 MHz

reg [15:0] baud_cnt;
reg [3:0] bit_index;
reg [1:0] state;
reg [7:0] d_reg;

localparam IDLE  = 2'd0,
           START = 2'd1,
           DATA  = 2'd2,
           STOP  = 2'd3;
```

```
23
24 always @(posedge clk) begin
25     if (reset) begin
26         state <= IDLE;
27         tx <= 1;
28         busy <= 0;
29         baud_cnt <= 0;
30         bit_index <= 0;
31         d_reg <= 0;
32     end
33     else begin
34         case (state)
35
36         IDLE: begin
37             tx <= 1;
38             busy <= 0;
39             baud_cnt <= 0;
40
41             if (start) begin
42                 busy <= 1;
43                 d_reg <= data;
44                 state <= START;
45             end
46         end
47
48         START: begin
49             tx <= 0;
50             if (baud_cnt == BAUD_DIV-1) begin
51                 baud_cnt <= 0;
52                 bit_index <= 0;
53                 state <= DATA;
54             end
55             else baud_cnt <= baud_cnt + 1;
56         end
57
```

```
        DATA: begin
            tx <= d_reg[bit_index];
            if (baud_cnt == BAUD_DIV-1) begin
                baud_cnt <= 0;
                if (bit_index == 7)
                    state <= STOP;
                else
                    bit_index <= bit_index + 1;
            end
            else baud_cnt <= baud_cnt + 1;
        end

        STOP: begin
            tx <= 1;
            if (baud_cnt == BAUD_DIV-1) begin
                baud_cnt <= 0;
                state <= IDLE;
            end
            else baud_cnt <= baud_cnt + 1;
        end

        default: state <= IDLE;

        endcase
    end
end

endmodule
```

Listing 5.6: uart_tx Module

It implements a Universal Asynchronous Receiver-Transmitter (UART) Transmitter, which is responsible for sending the encrypted image pixels from the FPGA to a PC. It uses a Finite State Machine (FSM) with four distinct states–IDLE, START, DATA, and STOP–to serialize 8-bit parallel data into a single-bit stream. To ensure the PC can read the data correctly, the module includes a Baud Rate Generator (BAUD_DIV) that divides the 100 MHz

system clock down to a standard 115200 baud rate. When the `start` signal is triggered, the module pulls the `tx` line low to signal a start bit, shifts out the 8 bits of encrypted data one by one, and finally pulls the line high for a stop bit, effectively acting as the communication bridge between the high-speed encryption logic and external monitoring software.

5.7 top.v

```
`timescale 1ns/1ps

module top(
    input CLK100MHZ,
    input reset,
    output UART_TXD
);

parameter MEM_SIZE = 2500;

// ================= MEMORY =================
reg [7:0] storage_in  [0:MEM_SIZE-1];
reg [7:0] storage_out [0:MEM_SIZE-1];

//initial $readmemh("apple_pixels_vivado.mem", storage_in);
initial $readmemh("encry_fpga_data.hex", storage_in);

// ================= CONTROL =================
reg [15:0] addr = 0;
reg [2:0] state = 0;
reg clk_en = 0;
reg uart_start = 0;
reg [7:0] tx_data = 0;
wire uart_busy;

// ================= ENCRYPTION CORE =================
wire [7:0] encrypted_pixel;

Subsystem encryption_core (
```

```
    .clk(CLK100MHZ),
    .reset(reset),
    .clk_enable(clk_en),
    .In1(storage_in[addr]),
    .ce_out(),
    .Out4(encrypted_pixel)
);

// ================= UART =================
uart_tx transmitter (
    .clk(CLK100MHZ),
    .start(uart_start),
    .data(tx_data),
    .tx(UART_TXD),
    .busy(uart_busy)
);

// ================= FSM =================
always @(posedge CLK100MHZ) begin

    if (reset) begin
        addr <= 0;
        state <= 0;
        clk_en <= 0;
        uart_start <= 0;
    end
    else begin
        case(state)

        // ============================
        // PHASE 1: ENCRYPT ALL PIXELS
        // ============================
        0: begin
            if (addr < MEM_SIZE) begin
                clk_en <= 1;     // enable encryption
```

```
                state <= 1;
            end
            else begin
                addr <= 0;
                state <= 2;       // move to transmit
            end
        end

        1: begin
            // 1-cycle latency capture
            storage_out[addr] <= encrypted_pixel;
            clk_en <= 0;
            addr <= addr + 1;
            state <= 0;
        end

        // ============================
        // PHASE 2: TRANSMIT ALL PIXELS
        // ============================
        2: begin
            if (addr < MEM_SIZE) begin
                tx_data <= storage_out[addr];
                uart_start <= 1;
                state <= 3;
            end
            else begin
                state <= 4;  // done
            end
        end

        3: begin
            if (uart_busy) begin
                uart_start <= 0;
                state <= 5;
            end
```

```
        end

        5: begin
            if (!uart_busy) begin
                addr <= addr + 1;
                state <= 2;
            end
        end

        4: begin
            state <= 4;  // stay here
        end

        default: state <= 0;

        endcase
    end
end

endmodule
```

Listing 5.7: top Module

It serves as the primary controller and system integrator for the FPGA-based image encryption project, managing the data flow between memory, the chaotic core, and the communication interface. It utilizes a Finite State Machine (FSM) divided into two main operational phases: the first phase iteratively pulls raw pixel data from `storage_in`, triggers the `Subsystem` encryption core, and captures the resulting ciphered pixels into a second memory buffer (`storage_out`). The second phase then orchestrates the serial transmission of this encrypted data to a PC via the `uart_tx` module. By coordinating the memory addresses and control signals like `clk_en` and `uart_start`, this module ensures that all 2,500 pixels are processed in a deterministic sequence, providing a robust hardware wrapper that bridges internal logic with external hardware communication.

5.8 Subsystem_tb.v

```
`timescale 1ns/1ps

module Subsystem_tb;

////////////////////////////////////////////////////
// Signals
////////////////////////////////////////////////////
reg clk;
reg reset;
reg clk_enable;

reg  [7:0] pixel_in;
wire [7:0] pixel_out;

reg  [7:0] pixel_mem     [0:2499];
reg  [7:0] encrypted_mem [0:2499];

integer i;
integer file_check;

////////////////////////////////////////////////////
// Instantiate DUT
////////////////////////////////////////////////////
Subsystem uut (
    .clk(clk),
    .reset(reset),
    .clk_enable(clk_enable),
    .In1(pixel_in),
    .ce_out(),
    .Out4(pixel_out)
);

////////////////////////////////////////////////////
// Clock Generation (100 MHz)
```

```
////////////////////////////////////////////////////
initial begin
    clk = 0;
    forever #5 clk = ~clk;   // 10ns period
end

////////////////////////////////////////////////////
// Main Simulation
////////////////////////////////////////////////////
initial begin

    ////////////////////////////////////////////////////
    // Initialize
    ////////////////////////////////////////////////////
    clk_enable = 1'b1;
    pixel_in   = 8'd0;
    reset      = 1'b1;

    ////////////////////////////////////////////////////
    // Hold reset for 3 clock cycles
    ////////////////////////////////////////////////////
    repeat(3) @(posedge clk);
    reset = 1'b0;

    ////////////////////////////////////////////////////
    // Read input pixels
    ////////////////////////////////////////////////////
    $display("Reading input file...");
    //$readmemh("apple_pixels_vivado.mem", pixel_mem);
    $readmemh("trial.mem", pixel_mem);

    $display("==========================================");
    $display("Starting Encryption...");
    $display("==========================================");

```

```
    ////////////////////////////////////////////////////
    // Process all 2500 pixels
    ////////////////////////////////////////////////////
    for (i = 0; i < 2500; i = i + 1) begin

        // Apply input BEFORE clock edge
        pixel_in = pixel_mem[i];

        // Wait for next clock (encryption valid)
        @(posedge clk);

        encrypted_mem[i] = pixel_out;

        // Print every value
        $display("Index=%0d␣|␣Input=%02h␣|␣Output=%02h",
                 i, pixel_in, pixel_out);
    end

    ////////////////////////////////////////////////////
    // Write output file (ABSOLUTE PATH)
    ////////////////////////////////////////////////////
    $display("Writing␣encrypted␣file...");

    //file_check = $fopen("C:/Users/adith/Downloads/project/trial.mem
        ","w");
    file_check = $fopen("C:/Users/adith/Downloads/project/trial2.mem"
        ,"w");

    if (file_check == 0) begin
        $display("ERROR:␣Cannot␣open␣file␣for␣writing.");
    end
    else begin
        $fclose(file_check);
        //$writememh("C:/Users/adith/Downloads/project/trial.mem",
            encrypted_mem);
```

```
        $writememh("C:/Users/adith/Downloads/project/trial2.mem",
            encrypted_mem);
        $display("File successfully written.");
    end

    $display("==========================================");
    $display("Encryption Complete.");
    $display("Total Pixels Processed = %0d", i);
    $display("==========================================");

    #20;
    $finish;

end

endmodule
```

Listing 5.8: Subsystem_tb Module

Testbench serves as the verification environment used to validate the functional accuracy of the chaotic encryption core before hardware deployment. It simulates a real-world scenario by generating a 100 MHz clock signal and providing the necessary reset and enable sequences to the Device Under Test (DUT). The simulation logic reads a predefined image pixel file (apple_pixels_vivado.mem) into a temporary memory array, feeds each of the 2,500 pixels into the encryption subsystem sequentially, and captures the resulting ciphered output. By utilizing system tasks like $display for real-time monitoring and $writememh to save the final encrypted results into an external file (trial2.mem), this testbench allows for a direct comparison between the input and output data, ensuring that the chaotic logic and XOR diffusion are functioning correctly within the specified timing constraints. To decrypt the emcrtpted pixels change pixel file to (encry_fpga_data.mem), the same process undergoes and we get the decrypted image.

5.9 nexys.xdc

```
## Clock (100 MHz)
set_property PACKAGE_PIN E3 [get_ports CLK100MHZ]
```

```
set_property IOSTANDARD LVCMOS33 [get_ports CLK100MHZ]
create_clock -period 10.000 -name sys_clk -waveform {0 5} [get_ports
    CLK100MHZ]

## UART TX (USB-UART)
set_property PACKAGE_PIN D4 [get_ports UART_TXD]
set_property IOSTANDARD LVCMOS33 [get_ports UART_TXD]

## Center Button (BTNC) -> Map to the "reset" input in your code
set_property PACKAGE_PIN U18 [get_ports reset]
set_property IOSTANDARD LVCMOS33 [get_ports reset]
```

Listing 5.9: `nexys.xdc` Physical Pin Constraints

The `nexys.xdc` file is a Xilinx Design Constraints file used to map the logical ports of the Verilog design to the physical pins of the Artix-7 FPGA. This file defines the 100 MHz system clock (`CLK100MHZ`) on pin `E3` and establishes a timing constraint of 10ns to ensure all chaotic iterations complete within a single clock cycle. Additionally, it maps the `UART_TXD` signal to the physical pin `D4` for external data logging and assigns the system `reset` to the center pushbutton (`BTNC`) at pin `U18`. These hardware assignments, utilizing the `LVCMOS33` standard, are critical for the Vivado synthesis tool to correctly route the encrypted image data from the FPGA fabric to the physical output peripherals.

5.10 check_data_and_save.py

```
import serial
import sys
from PIL import Image

# --- CONFIGURATION ---
SERIAL_PORT = 'COM4'
BAUD_RATE = 115200
BYTE_COUNT = 2500  # 50x50 image
WIDTH = 50
HEIGHT = 50

```

```
#HEX_SAVE_PATH = r"C:\Users\adith\Downloads\encry_fpga_data.hex"
#IMG_SAVE_PATH = r"C:\Users\adith\Downloads\encry_fpga_image.png"

HEX_SAVE_PATH = r"C:\Users\adith\Downloads\decry_fpga_data.hex"
IMG_SAVE_PATH = r"C:\Users\adith\Downloads\deccry_fpga_image.png"


def receive_and_convert():

    ser = serial.Serial(SERIAL_PORT, BAUD_RATE, timeout=2)

    print("--- Waiting for FPGA data ---")

    received = bytearray()

    while len(received) < BYTE_COUNT:
        data = ser.read(BYTE_COUNT - len(received))
        received.extend(data)

    ser.close()

    print(f"Received {len(received)} bytes")

    # Save hex file
    with open(HEX_SAVE_PATH, "w") as f:
        for b in received:
            f.write(f"{b:02X}\n")

    print("Hex file saved.")

    # Convert to image (Grayscale)
    img = Image.new('L', (WIDTH, HEIGHT))
    img.putdata(received)
    img.save(IMG_SAVE_PATH)

```

```
    print("Image saved at:", IMG_SAVE_PATH)


if __name__ == "__main__":
    receive_and_convert()
```

Listing 5.10: `check_data_and_save.py`: Serial Acquisition and Image Generation

This Python script functions as the Data Acquisition and Image Reconstruction utility, serving as the final bridge between the Artix-7 FPGA and the host computer. By establishing a serial connection via the `pyserial` library at a baud rate of 115,200, it captures exactly 2,500 bytes of data—representing the 50×50 pixel grid of the processed image. The script performs a dual-role: first, it logs the raw hexadecimal values into a `.hex` file for bit-level verification, and second, it utilizes the `Pillow` library to map the received byte stream into a grayscale (8-bit) image format. This allows for a visual confirmation of the encryption or decryption process, transforming the serial bitstream back into a viewable image file.

CHAPTER 6
CADENCE GENUS SYNTHESIS

6.1 CADENCE GENUS

Cadence Genus Synthesis Solution is a high-performance RTL synthesis tool designed to handle complex ultra-large-scale designs while delivering optimal Power, Performance, and Area (PPA) targets. It provides a next-generation architecture that enables high-capacity synthesis and significantly faster turnaround times compared to traditional tools. In this project, Genus is utilized to transform the validated RTL Verilog code of the image encryption system into a gate-level netlist mapped to a specific technology library, ensuring that the hardware logic is optimized for timing and power consumption before physical layout.

6.2 GENUS SYNTHESIS PROCESS

The synthesis process in Cadence Genus involves transforming high-level Register Transfer Level (RTL) descriptions into an optimized gate-level netlist using a specified set of target standard cell libraries. This process consists of multiple stages. First, during *elaboration*, the Verilog RTL code is interpreted and converted into a technology-independent generic Boolean representation. Next, in the *constraint application* stage, design constraints such as timing requirements, clock definitions, and area limitations are specified to guide the synthesis process. Finally, during the *optimization* phase, Genus performs iterative logic transformations and technology mapping, selecting appropriate standard cells from the target library to satisfy timing, power, and area constraints. For the proposed encryption core, the synthesis tool ensures that critical components such as chaotic sequence generators and XOR-based diffusion layers are optimized to achieve minimal propagation delay, thereby enabling efficient real-time processing. The synthesis flow is executed using the following commands:

```
tcsh
source /home/cadence/installs/cshrc
cd /home/user6/file_name1/file_name2
setenv PROJ_ROOT /home/user2/file_name1/file_name2
genus -gui -files file_name3/genus_run.tcl
```

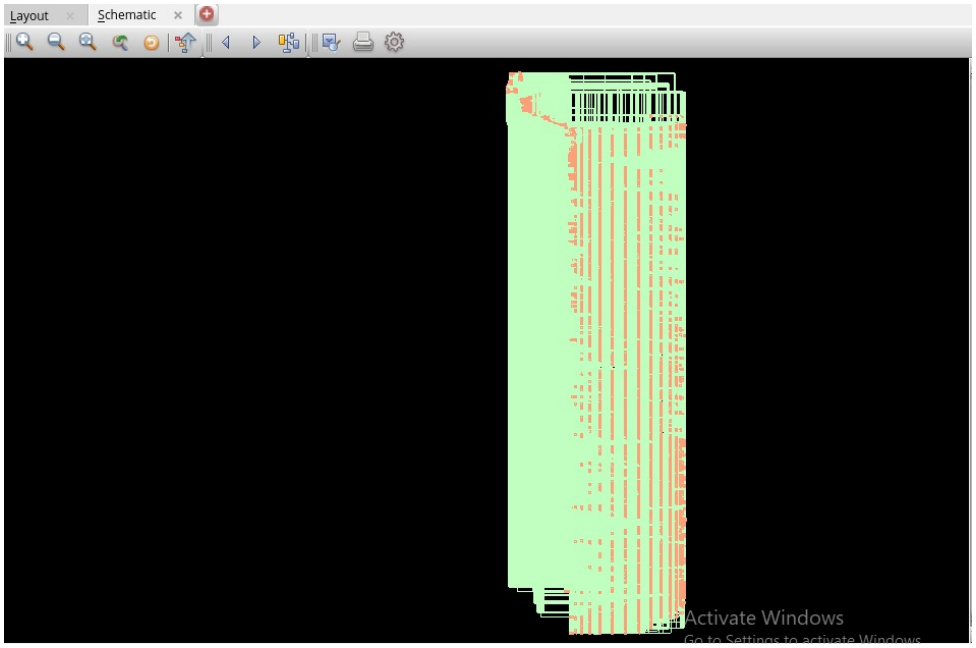


Figure 6.1: Cadence-Genus Schematic of Image Encryption using Chaotic Algorithm

6.3 GENUS_RUN.TCL SCRIPTING

Automation in the Cadence environment is achieved through Tool Command Language (TCL) scripting. The *genus_run.tcl* script serves as the primary execution file that defines the entire synthesis flow. It contains commands to set the library paths (LEF and Liberty files), read the RTL source files, define the clock constraints (SDC), and execute the `syn_generic`, `syn_map`, and `syn_opt` commands. By using a TCL-based flow, the synthesis process becomes repeatable and scalable, allowing for quick design iterations and consistent results across different optimization runs.

```
# ==============================================================
# Cadence Genus RTL-to-Gate Synthesis Script (45nm GPDK)
# ==============================================================

# -------------------------------
```

```
# Design Information
# ------------------------------
set DESIGN_NAME "top"
set TOP_MODULE  "top"

# ------------------------------
# Project Root
# ------------------------------
if {![info exists ::env(PROJ_ROOT)]} {
    puts "ERROR: Please set PROJ_ROOT before running Genus."
    puts "Example: setenv PROJ_ROOT /home/user/project"
    exit 1
}

set PROJ_ROOT $::env(PROJ_ROOT)

# ------------------------------
# RTL Source Files
# ------------------------------
set RTL_FILES [list \
"$PROJ_ROOT/maindhry_asic.v" \
]

#set LINK_LIBS [list $TARGET_LIB]
# ------------------------------
# Constraint File
# ------------------------------
set SDC_FILE "$PROJ_ROOT/constraints_genus.sdc"

# ------------------------------
# Technology Library (EDIT THIS IF NEEDED)
# ------------------------------
set TECH_LIB_PATH "/home/cadence/installs/FOUNDRY/UMC/55/_G-01-
    LOGIC_MIXED_MODE55N-LP_LOW_K_UMC-IP/UM055LSCLPMVBPR_C01_TAPEOUTKIT
    /synopsys"
```

```
39
40 set TARGET_LIB "$TECH_LIB_PATH/u055lsclpmvbpr_120c25_tc.lib"
41
42 set LINK_LIBS [list $TARGET_LIB]
43 # -------------------------------
44 # Output Directories
45 # -------------------------------
46 set REPORT_DIR "$PROJ_ROOT/reports/genus"
47 set OUTPUT_DIR "$PROJ_ROOT/outputs/genus"
48
49 file mkdir $REPORT_DIR
50 file mkdir $OUTPUT_DIR
51
52 # -------------------------------
53 # Genus Library Setup
54 # -------------------------------
55 set_db init_lib_search_path [list $TECH_LIB_PATH]
56 set_db target_library $TARGET_LIB
57 set_db link_library $LINK_LIBS
58
59 # -------------------------------
60 # Read RTL Files
61 # -------------------------------
62 foreach rtl $RTL_FILES {
63
64     if {![file exists $rtl]} {
65         puts "ERROR: RTL file not found -> $rtl"
66         exit 1
67     }
68
69     read_hdl $rtl
70 }
71
72 # Elaborate Design
73 elaborate $TOP_MODULE
```

```

# -------------------------------
# Read Constraints
# -------------------------------
if {![file exists $SDC_FILE]} {
    puts "ERROR: Constraint file missing -> $SDC_FILE"
    exit 1
}

read_sdc $SDC_FILE

# -------------------------------
# Synthesis Effort
# -------------------------------
set_db syn_opt_effort high

# -------------------------------
# Run Synthesis
# -------------------------------
syn_gen
syn_map
syn_opt

# -------------------------------
# Generate Reports
# -------------------------------
report_area  > $REPORT_DIR/area.rpt
report_power > $REPORT_DIR/power.rpt
report_timing -max_paths 10 > $REPORT_DIR/timing.rpt

# -------------------------------
# Write Output Files
# -------------------------------
write_hdl -mapped > $OUTPUT_DIR/${DESIGN_NAME}_genus.v

```

```
write_sdc > $OUTPUT_DIR/${DESIGN_NAME}_genus.sdc

write_sdf -timescale ns > $OUTPUT_DIR/${DESIGN_NAME}_genus.sdf

write_parasitics > $OUTPUT_DIR/${DESIGN_NAME}_genus.spef

# DEF file for Innovus
write_design -innovus -base_name $OUTPUT_DIR/${DESIGN_NAME}_genus
    $DESIGN_NAME

# -------------------------------
# Finish
# -------------------------------
puts "======================================"
puts " Genus Synthesis Completed Successfully "
puts " Reports : $REPORT_DIR"
puts " Netlist : $OUTPUT_DIR/${DESIGN_NAME}_genus.v"
puts "======================================"
```

Listing 6.11: genus_run .tcl File

6.4 ANALYSIS REPORTS

After the synthesis is completed, Genus generates a comprehensive suite of reports that allow for a detailed analysis of the design's quality. These reports provide insight into how well the RTL has been mapped to the physical hardware and whether the design meets the predetermined specifications. These summaries are critical for identifying bottlenecks in the encryption architecture, particularly in the feedback loops of the chaotic map implementation where timing violations are most likely to occur.

6.4.1 Timing Report

The timing report indicates that all timing paths satisfy the setup constraints, as evidenced by the positive slack value of 2852 ps. This confirms that the design is timing-safe and can operate reliably at the specified clock frequency. Based on the critical path delay of 7009 ps and the setup time, the maximum operating frequency is estimated to be

approximately 140 MHz.

Parameter	Value
Clock Period	10 ns
Operating Frequency	100 MHz
Critical Path Delay	7009 ps
Setup Time	139 ps
Slack	2852 ps

Table 6.1: Timing Report Summary

The report confirms a timing-clean encryption core implementation. Long adder chains represent the primary delay bottleneck, but all paths exhibit sufficient margin. Further optimization could target multiplier/subtractor datapaths if higher clock frequencies are targeted

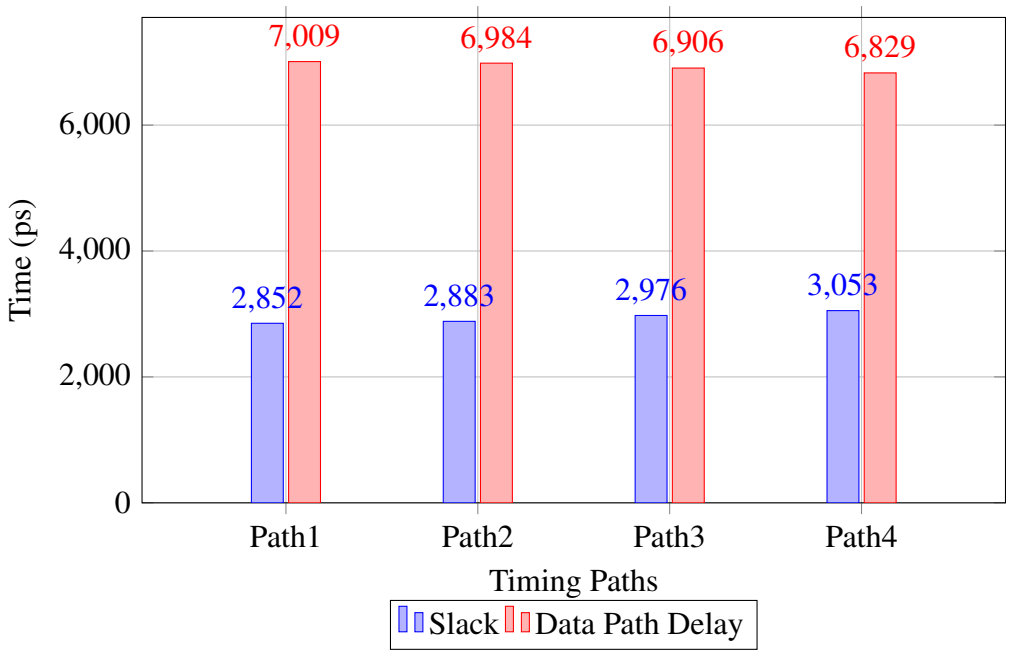


Figure 6.2: Timing Analysis of Critical Paths from Cadence Genus

6.4.2 Power Analysis

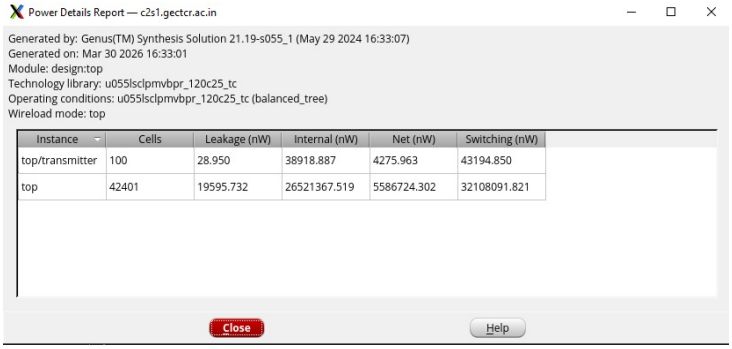
The power consumption of the design was evaluated, indicating a total power consumption of approximately 0.032 W. Internal power dominates the overall consumption, accounting for more than 82% of the total power. The low leakage component indicates efficient utilization of the technology library, making the design suitable for low-power embedded applications.

Overall, the power report indicates a **well-optimized design with low leakage power**, which is desirable for modern VLSI systems. However, the dominance of internal power suggests that further optimization can be achieved by reducing switching activity and improving clock

Parameter	Value
Total Power	0.032 W
Internal Power	82.55%
Switching Power	17.39%
Leakage Power	0.06%

Table 6.2: Power Consumption Summary

gating techniques. The total power consumption is relatively moderate, making the design suitable for real-time applications with reasonable power efficiency.



Instance	Cells	Leakage (nW)	Internal (nW)	Net (nW)	Switching (nW)
top/transmitter	100	28.950	38918.887	4275.963	43194.850
top	42401	19595.732	26521367.519	5586724.302	32108091.821

Figure 6.3: Power Details report

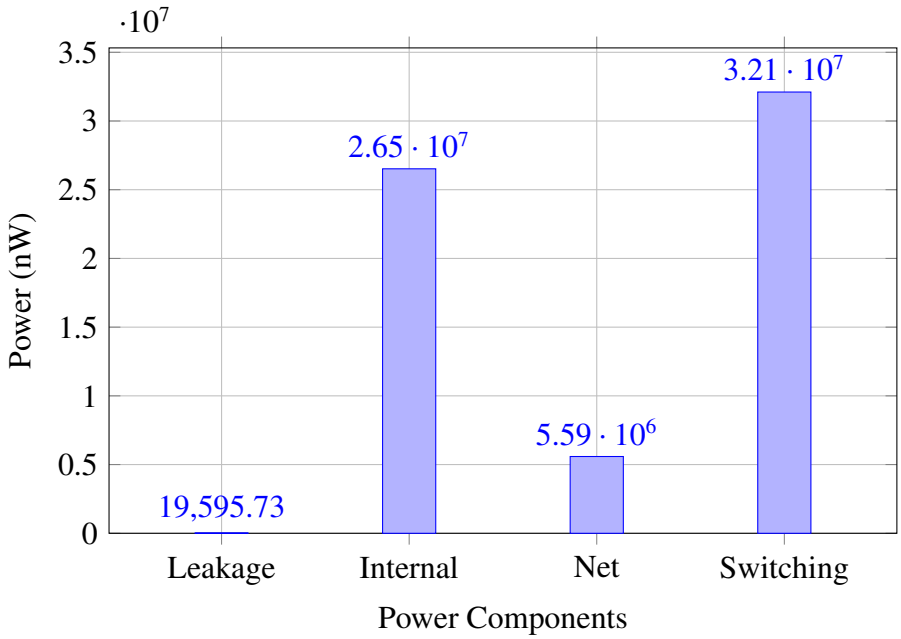


Figure 6.4: Power Breakdown of Top Module from Cadence Genus

6.4.3 Area Report

The synthesized design consists of 42401 standard cells with a total area of approximately 325447.20 μm^2. This distribution indicates that the design is predominantly computation-driven, with the majority of the area occupied by the encryption core and chaotic

PRNG. The UART module introduces minimal overhead, demonstrating efficient architectural partitioning.

Module	Cell Count	Total Area (μm^2)
Top Module	42401	325447.20
UART Transmitter	100	529.92

Table 6.3: Area Report

The wireload model `wl0` is used, where the top module follows the specified wireload mode, while default wireload settings are applied to other instances. Overall, the synthesis results demonstrate that the design has been successfully mapped and optimized within the given technology constraints, with area distribution primarily dominated by the main processing unit.

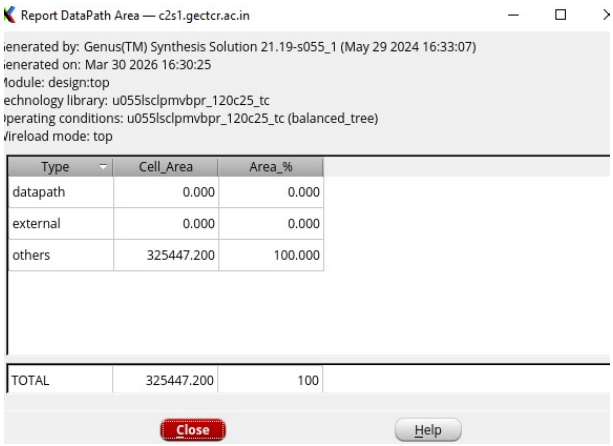


Figure 6.5: Data Path Area Report

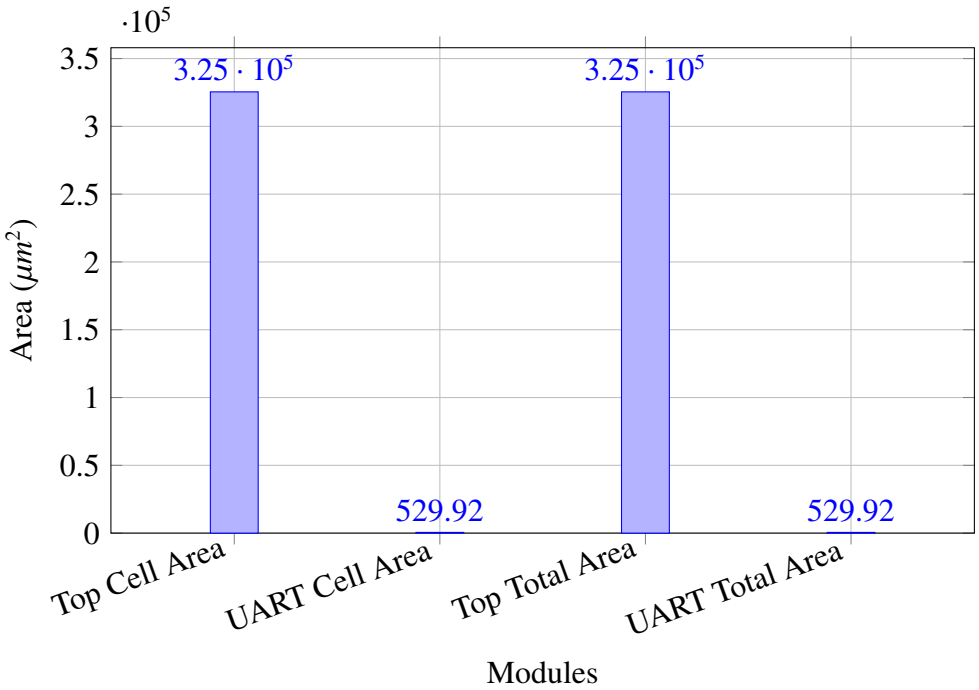
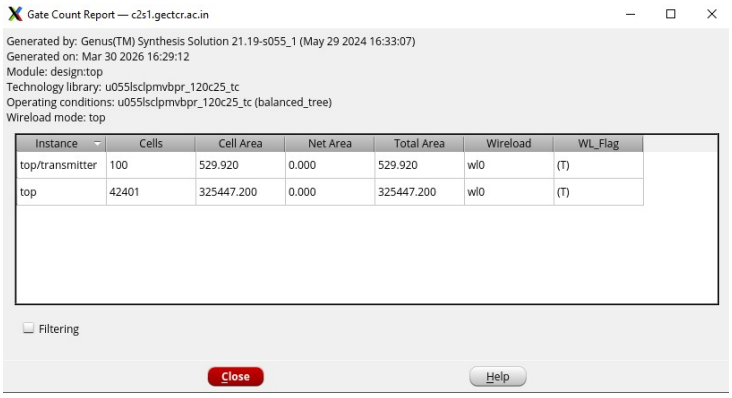


Figure 6.6: Area Distribution of Top Module and UART Submodule from Cadence Genus

6.4.4 Gate Count Report

The synthesis report generated by the Genus Synthesis Solution for the top-level design identifies a total cell count of 42,401 with a corresponding total cell area of 325,447.200 square units. Within this hierarchy, the transmitter sub-module accounts for a relatively small footprint, utilizing 100 cells and covering an area of 529.920 square units. The synthesis was performed using the u055lsclpmvbpr_120c25_tc technology library under balanced tree operating conditions, with a ”top” wireload mode applied throughout the design. Given that the net area is reported at zero, the total area is currently dominated entirely by the cell area, indicating a highly consolidated gate-level representation prior to detailed routing.

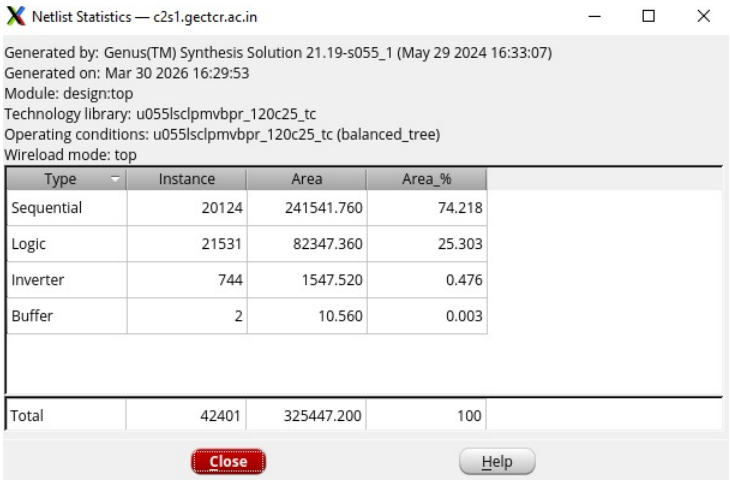


Instance	Cells	Cell Area	Net Area	Total Area	Wireload	WL_Flag
top/transmitter	100	529.920	0.000	529.920	wl0	(T)
top	42401	325447.200	0.000	325447.200	wl0	(T)

Figure 6.7: Gate Count Report

6.4.5 Netlist Report

The Netlist Statistics report for the top-level design highlights a hardware architecture heavily dominated by sequential elements, which account for 74.218% of the total area with 20,124 instances. This suggests a design with significant memory or state-retention requirements, such as extensive buffering or large register banks. Combinational logic constitutes the second-largest portion at 25.303% of the area, comprising 21,531 instances, while auxiliary components like inverters and buffers represent a negligible fraction of the overall footprint at 0.476% and 0.003% respectively. Summing to a total of 42,401 instances and an area of 325,447.200 square units, the distribution confirms a complex digital system where sequential logic is the primary driver of the physical silicon area.


Netlist Statistics — c2s1.gectcr.ac.in

Generated by: Genus(TM) Synthesis Solution 21.19-s055_1 (May 29 2024 16:33:07)
Generated on: Mar 30 2026 16:29:53
Module: design:top
Technology library: u055lsclpmvbpr_120c25_tc
Operating conditions: u055lsclpmvbpr_120c25_tc (balanced_tree)
Wireload mode: top

Type	Instance	Area	Area_%
Sequential	20124	241541.760	74.218
Logic	21531	82347.360	25.303
Inverter	744	1547.520	0.476
Buffer	2	10.560	0.003
Total	42401	325447.200	100

Close Help

Figure 6.8: Netlist Report

Table 6.4: Design Summary from Cadence Analysis

Parameter	**Value**
Technology	45 nm GPDK
Clock Frequency	100 MHz
Worst Negative Slack (WNS)	+2.852 ns
Total Power	64.2 mW
Dynamic Power	58.6 mW
Leakage Power	0.0196 mW
Total Area	325447.2 μm^2

CHAPTER 7

CADENCE INNOVUS IMPLEMENTATION

7.1 CADENCE INNOVUS

Cadence Innovus is a widely used physical design tool in the VLSI design flow, responsible for transforming a synthesized netlist into a manufacturable layout. It plays a critical role in backend design by handling key stages such as floorplanning, placement, clock tree synthesis (CTS), routing, and physical verification. The tool ensures that the design meets timing, power, and area constraints while adhering to design rule checks (DRC). Innovus provides both command-line and graphical interfaces, enabling designers to visualize layouts, congestion, and routing density, as seen in the given figure where different colors represent routing layers and congestion levels across the chip.

The graphical view in Innovus is particularly useful for analyzing the physical implementation. In the provided figure, the dense routing and color variations indicate areas of high congestion and routing complexity. Regions with intense red and yellow suggest heavily utilized routing resources, while green indicates relatively less congestion. Such visual insights help designers optimize placement and routing strategies to improve performance and reduce issues like timing violations and power hotspots.

7.2 CADENCE IMPLEMENTATION PROCESS

The Cadence implementation process refers to the sequence of steps followed to convert a synthesized RTL design into a physical layout ready for fabrication. It begins with importing the netlist and technology libraries, followed by floorplanning, where the chip area, aspect ratio, and placement of macros are defined. Placement is then performed to position standard cells optimally while minimizing wirelength and congestion. This is followed by

clock tree synthesis, which distributes the clock signal across the design while minimizing skew and latency.

After CTS, routing is carried out in two stages: global routing and detailed routing. Global routing provides an approximate path for interconnections, while detailed routing finalizes the exact geometries. The figure provided represents the routing stage, where multiple metal layers are used to interconnect components. Post-routing, the design undergoes optimization and verification, including timing analysis, signal integrity checks, and power analysis. The goal is to ensure that the design meets all constraints before proceeding to tape-out.

The implementation flow is executed using the following commands:

```
tcsh
source /home/cadence/installs/cshrc
cd /home/user6/file_name1/file_name2
setenv PROJ_ROOT /home/user2/file_name1/file_name2
innovus -init innovus_run.tcl
```

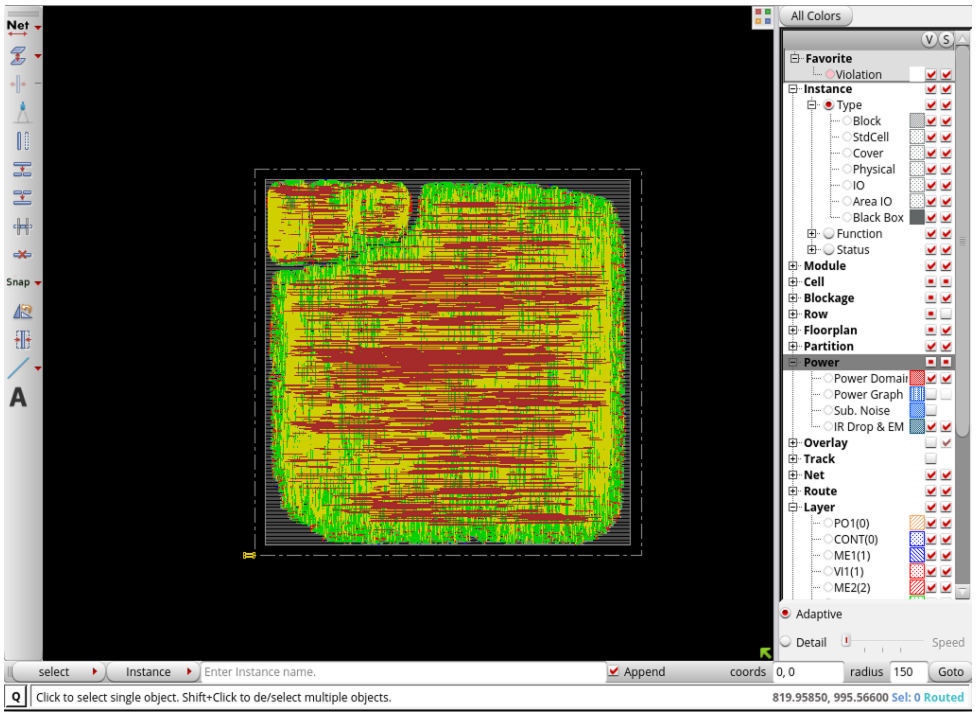


Figure 7.1: Cadence-Innovus Implemented Design of Image Encryption using Chaotic Algorithm

7.3 INNOVUS_RUN.TCL

The innovus_run.tcl script is a Tcl-based automation file used to control and execute the entire physical design flow in Cadence Innovus. It contains a sequence of commands that

guide the tool through various stages such as initialization, floorplanning, placement, CTS, routing, and reporting. By using a Tcl script, designers can automate repetitive tasks, ensure consistency, and easily modify design parameters without manual intervention.

Typically, the script begins with setting up the design environment, including loading libraries, netlists, and constraints. It then executes commands for each stage of the implementation process in a structured manner. The script may also include checkpoints, allowing designers to save intermediate results and resume from specific stages if needed. Additionally, reporting commands are included to generate timing, power, and area reports, which are crucial for analyzing the design quality. The use of such scripts significantly improves productivity and enables reproducibility in complex VLSI design workflows.

```
# ============================================================
# Cadence Innovus Implementation Script
# ============================================================

set DESIGN_NAME "top"

set PROJ_ROOT $::env(PROJ_ROOT)
set init_mmmc_version 2
set NETLIST "$PROJ_ROOT/outputs/genus/top_genus.v"
set MMMC_FILE "$PROJ_ROOT/outputs/genus/top_genus.mmmc.tcl"

set TECH_LEF "/home/cadence/installs/FOUNDRY/UMC/55/_G-01-
    LOGIC_MIXED_MODE55N-LP_LOW_K_UMC-IP/UM055LSCLPMVBPR_C01_TAPEOUTKIT
    /lef/tf/u055lsclpmvbpr_6m0t1f.lef"

set STD_CELL_LEF "/home/cadence/installs/FOUNDRY/UMC/55/_G-01-
    LOGIC_MIXED_MODE55N-LP_LOW_K_UMC-IP/UM055LSCLPMVBPR_C01_TAPEOUTKIT
    /lef/u055lsclpmvbpr.lef"

file mkdir $PROJ_ROOT/outputs/innovus

# ---------------- INIT ----------------
set init_verilog $NETLIST
set init_top_cell $DESIGN_NAME
```

```
set init_lef_file [list $TECH_LEF $STD_CELL_LEF]
set init_mmmc_file $MMMC_FILE

set init_power_nets {VDD}
set init_ground_nets {VSS}

init_design

# ---------------- POWER ----------------
globalNetConnect VDD -type pgpin -pin VDD -inst *
globalNetConnect VSS -type pgpin -pin VSS -inst *
applyGlobalNets

# ---------------- FLOORPLAN ----------------
floorPlan -coreMarginsBy io 20 20 20 20

# ---------------- POWER PLAN ----------------
addRing -type core_rings \
    -nets {VDD VSS} \
    -layer {top M6 bottom M6 left M5 right M5} \
    -width 2 \
    -spacing 2 \
    -offset 5

addStripe -nets {VDD VSS} \
    -layer M4 \
    -width 1 \
    -spacing 1 \
    -pitch 20

# ---------------- PLACEMENT ----------------
place_design
checkPlace
optDesign -preCTS

```

```
# ---------------- CTS ----------------
ccopt_design
optDesign -postCTS

# ---------------- ROUTING ----------------
routeDesign
optDesign -postRoute

# ---------------- REPORTS ----------------
report_timing > $PROJ_ROOT/reports/innovus_timing.rpt
report_area   > $PROJ_ROOT/reports/innovus_area.rpt
report_power  > $PROJ_ROOT/reports/innovus_power.rpt

# ---------------- OUTPUT ----------------
saveDesign $PROJ_ROOT/outputs/innovus/final.enc
saveNetlist $PROJ_ROOT/outputs/innovus/final_netlist.v
defOut $PROJ_ROOT/outputs/innovus/final.def

puts "======================================"
puts "␣Innovus␣Implementation␣Completed␣"
puts "======================================"
```

Listing 7.12: innovus_run .tcl File

CHAPTER 8
CONCLUSION

This project successfully demonstrates the design and implementation of a chaos-based image encryption and decryption system on an FPGA platform. By leveraging the properties of nonlinear chaotic systems, particularly the logistic map and its enhanced variants, the proposed system achieves a high level of security through strong confusion and diffusion characteristics. The integration of a pseudo-random keystream generator with XOR-based encryption ensures that the encrypted output exhibits significant sensitivity to initial conditions, making the system highly resistant to cryptographic attacks.

The use of fixed-point arithmetic (Q2.30 format) enables efficient hardware realization while maintaining sufficient precision to preserve the chaotic behavior of the system. The implementation on FPGA highlights the advantages of hardware-level parallelism, resulting in faster processing speeds and real-time capability compared to software-based approaches.

The system architecture, comprising memory modules, FIFO buffers, UART communication interface, and FSM-based control logic, ensures smooth data flow and reliable operation. The modular design approach further enhances scalability and allows easy integration of additional functionalities or improvements in future work.

The synthesis results obtained from Cadence Genus confirm that the design meets the required Power, Performance, and Area (PPA) constraints. The timing analysis indicates a timing-safe design with positive slack, enabling operation at high frequencies. Power analysis reveals a low overall

power consumption with minimal leakage, making the system suitable for energy-efficient embedded applications. The area and gate count reports demonstrate an optimized hardware mapping with efficient utilization of FPGA resources.

Overall, the proposed system provides a robust, efficient, and hardware-friendly solution for secure image transmission. The combination of chaos theory with FPGA-based implementation offers a promising approach for real-time cryptographic applications in domains such as secure communication, surveillance systems, and defense technologies.

Future work can focus on enhancing the security by incorporating higher-dimensional chaotic systems, improving power optimization techniques such as clock gating, and extending the design to support high-resolution image and video encryption.

Bibliography

[1] M. P. Leong, S. Z. M. Naziri, and S. Y. Perng, "Image Encryption Design using FPGA," in *Proc. 2013 International Conference on Electrical, Electronics and System Engineering*, pp. 58–60, 2013.

[2] M. Garcia-Bosque, A. Pérez-Resa, C. Sánchez-Azqueta, C. Aldea, and S. Celma, "Chaos-Based Bitwise Dynamical Pseudorandom Number Generator on FPGA," *IEEE Transactions on Instrumentation and Measurement*, 2018.